NPTEL Week 2 – Autoencoders: Complete Exam Notes

Course: Fundamentals of Generative AI and Large Language Models: Theory and Practice. Covers Lectures 10–16: basics, loss functions, CNN/shallow/deep autoencoders, denoising, sparse, contractive, numerical example, limitations (why VAEs).

0. Roadmap of the week

Lecture	Topic
10	Intro, architecture, undercomplete vs overcomplete
11	Linear AE, activations, MSE and BCE loss, tied weights
12	Types: CNN, shallow, deep AE; upsampling and transposed convolution
13	Denoising AE
14	Sparse AE (KL-divergence penalty)
15	Contractive AE (Jacobian penalty)
16	Full numerical example (5→4→3→4→5) and limitations of AEs

One-line summary of the whole week: an autoencoder is a neural network trained to reproduce its own input through a bottleneck, so that the bottleneck (latent code) becomes a useful compressed representation. Plain AEs have weaknesses (identity copying, no guarantee of useful features, cannot generate), which regularized AEs (denoising, sparse, contractive) and finally VAEs address.

---

1. From supervised learning to autoencoders

- Supervised learning: data comes as (X, Y). The model learns X → Y. ANNs, CNNs, RNNs are mostly used this way (e.g. Iris: features → class label).

- Unsupervised learning: only X is available (e.g. age, height, weight with no label). The model must find hidden patterns, structure and correlations in X.
- Representation learning is a special category of unsupervised deep learning that aims to learn meaningful representations of the data itself.
- Autoencoders are exactly such models: they learn representations by reconstructing their own inputs. The key trick: the input itself is the target. No labels needed.

2. Architecture of an autoencoder

Three components:

1. Encoder: maps input x to a smaller internal representation.
2. Latent code / bottleneck (h or z): the compressed representation.
3. Decoder: reconstructs the input from the latent code, giving $\hat{x}$.

Two objectives:

- Representation learning (encoder): learn a compact latent representation, enabling unsupervised feature learning.
- Reconstruction learning (decoder): reconstruct the original input from the latent representation.

Flow: x → Encoder → h → Decoder → $\hat{x}$, with training goal $\hat{x} \approx x$.

An autoencoder is a feed-forward neural network that learns to reconstruct its own input.

2.1 Encoder objective: compression (100 → 75 → 25 example)

- Input has 100 features.
- Hidden layer 1 has 75 neurons. Fully connected, so each neuron sees all 100 inputs and gives one value → 75 values.
- Hidden layer 2 has 25 neurons → 25 values. This is the latent representation h.
- Dimension path: 100 → 75 → 25.
- Because the network is forced to squeeze, it must keep only the most important information. That is how representation learning is achieved.

2.2 Decoder objective: reconstruction (25 → 75 → 100)

- The decoder takes the 25-dim latent code, expands to 75, then to 100 (same as the original input size).

- It does not copy the input. It only has the compressed information in h to work with.
- If h captured the important structure, $\hat{x}$ is close to x. That achieves reconstruction learning.

Exam point: the number of output neurons always equals the number of input features.

3. Undercomplete vs overcomplete autoencoders

Let dim(h) be hidden (latent) size and dim(x) be input size.

	Undercomplete	Overcomplete
Condition	dim(h) < dim(x)	dim(h) ⩾ dim(x)
Example	6 → 3 → 6	4 → 6 → 4
Behaviour	Forced to learn the most important features	Can learn a trivial identity (copy x into h, copy h into $\hat{x}$)
Risk	Information loss if too small	Overfitting, poor generalization, no meaningful features
Fix	Choose size carefully	Regularize (denoising, sparse, contractive)

Loss-free encoding: if we are still able to reconstruct $\hat{x}$ perfectly from h in an undercomplete AE, then h is a loss-free encoding of x; it captured all the important characteristics of x.

Why overcomplete is a problem: reconstruction error can be near zero with no useful learning. Regularized variants (Lectures 13–15) prevent this trivial identity mapping.

---

4. The linear autoencoder: shapes and tied weights

Encoder: $H = g(X \cdot We)$ Decoder: $\hat{X} = f(H \cdot Wd)$

- X = input matrix, H = encoded data, $\hat{X}$ = reconstruction.
- We = encoder matrix, Wd = decoder matrix.
- g(·) = encoder activation, f(·) = output (decoder) activation.

4.1 Shape bookkeeping (the lecture example)

Suppose 1000 samples, 6 features, latent size 3.

Matrix	Shape	Reason
X	1000 × 6	samples × features
We	6 × 3	maps R^6 to R^3
H = X·We	1000 × 3	latent codes
Wd	3 × 6	maps R^3 back to R^6
X̂ = H·Wd	1000 × 6	same as X

Rule: inner dimensions must match; the decoder matrix has the reverse shape of the encoder matrix.

4.2 Generic setting vs tied weights

- Generic setting: We and Wd are both learned independently. More parameters (6×3 + 3×6 = 36 weights here), longer training.
- Tied-weights assumption: $W_d = W_e^T$. Only We is learned. W_e^T is 3 × 6, which has exactly the right shape. Parameters halve (18 here). Common and computationally efficient.

With tied weights, writing W for We: $\hat{X} = H \cdot W^T = X \cdot W \cdot W^T$

5. Choice of activation functions

The decoder output must match the range of the input data.

Input type	Decoder activation f	Loss	Why
Binary, x ∈ {0,1}	Sigmoid, f(a) = 1/(1+e^(−a))	Binary cross-entropy	Output in (0,1), can be read as probability
Scaled to [0,1] (e.g. normalized pixels)	Sigmoid	BCE (commonly)	Keeps output in valid range
Real-valued, unscaled/standardized	Linear, f(a) = a	MSE	Output can take any real value

Encoder activation g: for binary inputs sigmoid or ReLU; for real-valued inputs commonly ReLU, sigmoid or tanh.

Shortcut from the lecture margin note: linear → MSE; sigmoid → BCE.

Hidden layers in the numerical example use ReLU, ReLU(x) = max(0, x). The output layer is linear because inputs are continuous.

---

6. Reconstruction loss

6.1 Mean Squared Error (real-valued inputs)

Per-sample loss: $l_i = \|x_i - \hat{x}_i\|^2$

Total loss over m samples: $L = (1/m)\ \Sigma\ l_i$.

In matrix form for the tied-weight linear AE (W is the learnable quantity):

$f(W) = \min_W \|X - X\,W\,W^T\|^2$ (reconstruction error)

With regularization: $f(W) = \min_W \|X - X\,W\,W^T\|^2 + \lambda\|W\|^2$

The $\lambda\|W\|^2$ term (regularization term) keeps the weights small to avoid overfitting.

Derivation chain to remember: $\hat{X} = H \cdot Wd$, with $Wd = W^T$ and $H = X \cdot We$, $We = W$, gives $\hat{X} = X \cdot W \cdot W^T$.

After training we get the optimized W*:

- Encoder: $H^* = X \cdot W^*$
- Decoder: $\hat{X} = H^* \cdot W^{*T}$
- $\|X - \hat{X}\|^2$ is small, so $\hat{X}$ is a close approximation of X.

- H* captures the most informative structure needed for reconstruction, not an exact copy of the input.

MSE for a single sample with d features (the slide's numerical form): MSE = (1/d) Σ $(x_i - \hat{x}_i)^2$.

6.2 Binary Cross-Entropy (binary inputs)

For sample i with d features (j indexes features):

$l_i = - \Sigma_{j=1..d} [x_{ij} \log(\hat{x}_{ij}) + (1 - x_{ij}) \log(1 - \hat{x}_{ij})]$

Total: $L_{BCE} = (1/n) \Sigma_i \Sigma_j [...]$ with the minus sign in front.

With regularization: $f(W) = \min_W [L_{BCE}(X, \hat{X}) + \lambda \|W\|^2]$.

Why the minus sign: $\hat{x}$ is between 0 and 1, so $\log(\hat{x})$ is negative (and 0 at $\hat{x} = 1$). The bracket is negative; the minus makes the loss positive.

How it behaves: if x = 1, only the first term is active: loss = $-\log(\hat{x})$, small when $\hat{x}$ near 1. If x = 0, only the second: loss = $-\log(1 - \hat{x})$, small when $\hat{x}$ near 0.

Worked example: BCE

x = [1, 0, 1, 1, 0], $\hat{x}$ = [0.92, 0.08, 0.81, 0.76, 0.12]. Use natural log.

j	x	term	value
1	1	$-\ln(0.92)$	0.0834
2	0	$-\ln(1-0.08) = -\ln(0.92)$	0.0834
3	1	$-\ln(0.81)$	0.2107
4	1	$-\ln(0.76)$	0.2744
5	0	$-\ln(1-0.12) = -\ln(0.88)$	0.1278

Sample loss l_i = 0.0834 + 0.0834 + 0.2107 + 0.2744 + 0.1278 = 0.7797 (about 0.78). Averaged per feature: 0.156.

Notice the least accurate output (0.76 where the target is 1) contributes the largest term.

Worked example: MSE

x = [1.0, 2.0, 3.0], $\hat{x}$ = [0.8, 2.5, 2.9]. Errors: 0.2, −0.5, 0.1. Squares: 0.04, 0.25, 0.01. Sum = 0.30. MSE = 0.30/3 = 0.10.

MSE vs BCE summary

	MSE	BCE
Data	continuous / real	binary or in [0,1]
Output activation	linear	sigmoid
Penalty	squared difference	log-loss, penalizes confident wrong outputs heavily

7. Complete learning flow

Tabular data: encoder and decoder are commonly multilayer perceptrons (MLPs). X (n samples × 10 features) → Encoder → Z (n × 2, latent space) → Decoder → X′ (n × 10). Loss: $L(X, X') = \|X - X'\|^2$; trained by minimizing the difference between input and reconstruction.

Images: the encoder uses convolutional layers to extract spatial features and the decoder uses upsampling / transposed-convolution layers to reconstruct the image (see next section).

Training in one sentence: adjust all weights and biases so that the reconstruction becomes closer and closer to the original input (gradient descent on the reconstruction loss).

---

8. Types of autoencoders (Lecture 12)

8.1 Shallow autoencoder

- Simplest form: only ONE hidden layer between input and output, hence "shallow".
- Architecture: Input $X \in R^n$ → Hidden (latent) $z \in R^m$ with $m < n$ → Output $\hat{X} \in R^n$.
- Keras code (MNIST, 784 inputs):

```
inputs = Input(shape=(784,))
encoder_output = Dense(32, activation='relu')(inputs)       # latent, 32
decoder_output = Dense(784, activation='sigmoid')(encoder_output)
```

- 784 → 32 → 784. Why 784? A 28×28 image is flattened (28 × 28 = 784).
- Output activation rule: pixels scaled to [0,1] → sigmoid + binary cross-entropy. Unscaled or standardized real values → linear + MSE.
- Parameter count (including biases): encoder 784×32 + 32 = 25,120; decoder 32×784 + 784 = 25,872; total 50,992.

8.2 Deep autoencoder

- Extension of the shallow AE: multiple hidden layers in both encoder and decoder.
- Learns hierarchical, abstract representations.
- Architecture: X → h1 → h2 → … → z (m < n) → … → X̂.
- Encoder progressively compresses; latent space captures high-level features; decoder progressively reconstructs.
- Code from lecture (symmetric 784 → 128 → 64 → 32 → 64 → 128 → 784):

```
x = Dense(128, activation='relu')(inputs)
x = Dense(64,  activation='relu')(x)
latent = Dense(32, activation='relu')(x)      # latent space (32)
x = Dense(64,  activation='relu')(latent)
x = Dense(128, activation='relu')(x)
reconstruction = Dense(784, activation='sigmoid')(x)
```

- The decoder mirrors the encoder (symmetric architecture is typical).

8.3 Convolutional (CNN) autoencoder

Designed for image data.

- Encoder: convolution + pooling layers reduce the spatial size while extracting features.
- Latent space: output of the final convolutional layer in the encoder = compressed representation.
- Decoder: upsampling (or transposed convolution) layers bring the data back to the original size.

Lecture architecture on MNIST (28×28×1 input):

Stage	Layer	Output shape
Input	–	28×28×1
enc_conv1	Conv2D(64, 3×3, relu, same)	28×28×64
enc_pool1	MaxPooling2D(2×2)	14×14×64
enc_conv2	Conv2D(128, 3×3, relu, same)	14×14×128
enc_pool2	MaxPooling2D(2×2)	7×7×128
latent	Conv2D(256, 3×3, relu, same)	7×7×256
dec_up1	UpSampling2D(2×2)	14×14×256
dec_conv1	Conv2D(128, 3×3, relu, same)	14×14×128
dec_up2	UpSampling2D(2×2)	28×28×128
dec_conv2	Conv2D(64, 3×3, relu, same)	28×28×64
output	Conv2D(1, 3×3, sigmoid, same)	28×28×1

Key points:

- padding='same' keeps spatial size after convolution; only pooling halves it (28 → 14 → 7).
- Last layer: 1 filter (to get back one channel), sigmoid so pixels lie in [0,1] (matching normalized input).
- A 3×3 convolution looks at the local neighborhood of every pixel, which is how the model learns neighbor dependencies.
- Note on the slide code: the function returns a variable named `bottleneck` while the latent tensor is named `latent`. Use `latent` consistently in your own code, otherwise it throws a NameError.
- Honest observation: 7×7×256 = 12,544 numbers, far more than the 784 input pixels. In element count this latent is overcomplete; the compression is in spatial size only. This is why CNN AEs are usually paired with regularization or smaller channel counts if you want real compression.

8.4 DNN (dense) AE vs CNN AE

	DNN autoencoder	CNN autoencoder
Input handling	Flattens image (28×28 → 784)	Keeps 2-D structure
Spatial relationships	Fails to preserve edges, contours, shapes	Preserves local spatial structure
Learns	Global dense mappings	Edges, textures, shapes
Reconstruction (Fashion-MNIST demo)	Blurry, loses detail	Sharper, finer detail

8.5 Upsampling (decoder building block)

Problem: after the encoder, 28×28 became 7×7, but the decoder must reconstruct 28×28. We need to increase spatial resolution: upsampling. Plain upsampling has NO learnable parameters.

Output size = input size × S (upsampling factor).

Nearest-neighbor upsampling (S = 2): every element is copied into a 2×2 block.

```
[1 2]        [1 1 2 2]
[3 4]   =>   [1 1 2 2]
             [3 3 4 4]
             [3 3 4 4]
```

Bed-of-nails (zero insertion): each element goes to the top-left of its 2×2 block, rest zeros.

```
[1 2]        [1 0 2 0]
[3 4]   =>   [0 0 0 0]
             [3 0 4 0]
             [0 0 0 0]
```

8.6 Transposed convolution (learnable upsampling)

Also called deconvolution. Each input value multiplies the whole kernel, and the scaled kernels are placed at shifted positions and added (overlaps sum).

Output size (stride 1, no padding) = (Input + Kernel − 1). General formula with stride s and padding p: (n − 1)·s + k − 2p.

Worked example

Input = [[1, 2],[0, 1]], Kernel = [[4, 3],[2, 1]]. Output size = 2 + 2 − 1 = 3 → 3×3.

Input value	Scaled kernel placed at	Matrix
1 (position 0,0)	top-left	[[4,3,0],[2,1,0],[0,0,0]]
2 (position 0,1)	shift right by 1	[[0,8,6],[0,4,2],[0,0,0]]
0 (position 1,0)	shift down by 1	all zeros
1 (position 1,1)	shift down 1, right 1	[[0,0,0],[0,4,3],[0,2,1]]

Add all four:

```
[4  11  6]
[2   9  5]
[0   2  1]
```

Check the centre: 1 + 4 + 4 = 9 (three kernels overlap there). Corner cells get only one contribution. Uneven overlap is exactly what causes the checkerboard problem.

Checkerboard artifacts

- Unwanted grid-like patterns in images from transposed convolutions: alternating bright/dark pixels, regular square patterns, uneven texture in flat regions.
- Not random noise; a systematic artifact of the operation (uneven kernel overlap).

8.7 Upsampling + convolution (the fix)

First upsample, then convolve with the kernel. Upsampling spreads values uniformly, and the following convolution applies the kernel evenly, avoiding uneven overlap.

Worked example

Input [[1,2],[0,1]] → nearest-neighbor upsample →

```
[1 1 2 2]
[1 1 2 2]
[0 0 1 1]
[0 0 1 1]
```

Kernel [[4,3],[2,1]], stride 1, no padding → 3×3 output. Slide each 2×2 window and take the elementwise product sum:

Window	Computation	Result
[[1,1],[1,1]]	4+3+2+1	10
[[1,2],[1,2]]	4+6+2+2	14
[[2,2],[2,2]]	8+6+4+2	20
[[1,1],[0,0]]	4+3+0+0	7
[[1,2],[0,1]]	4+6+0+1	11
[[2,2],[1,1]]	8+6+2+1	17
[[0,0],[0,0]]	0	0
[[0,1],[0,1]]	0+3+0+1	4
[[1,1],[1,1]]	4+3+2+1	10

```
[10 14 20]
[ 7 11 17]
[ 0  4 10]
```

	Upsampling	Transposed conv	Upsampling + conv
Learnable parameters	No	Yes	Yes (the conv part)
Checkerboard problem	n/a	Yes	Avoided

8.8 Latent dimensionality controls reconstruction quality

- Too small (e.g. 2-D latent on MNIST): model discards important details, blurry/poor reconstruction.
- Larger (e.g. 5-D): reconstruction improves and looks close to ground truth.
- Beyond a point: diminishing returns.

---

9. Regularization techniques in autoencoders

Why needed: with large hidden layers (overcomplete), a plain AE learns the identity mapping and overfits. A regularized AE prevents the trivial copy and forces meaningful feature learning. Three techniques: denoising (corrupt input), sparse (penalize active neurons), contractive (penalize sensitivity).

9.1 Denoising autoencoder (DAE), Lecture 13

Idea: corrupt the input randomly, then train the network to reconstruct the ORIGINAL clean input.

Pipeline: x (clean) → corruption → $\tilde{x}$ (noisy) → Encoder → h → Decoder → $\hat{x}$, with $\hat{x} \approx x$ (NOT $\tilde{x}$).

Formulation step by step

1. Corrupt: $\tilde{x} = \text{corruption}(x)$.
2. Encode: $h = g(W\tilde{x} + b)$.
3. Decode: $\hat{x} = f(W' h + c)$.
4. Pipeline: $\tilde{x} \rightarrow \hat{x} \approx x$.
5. Objective: min $L(x, \hat{x})$. MSE for continuous data, BCE for binary or normalized images.
6. Neighborhood connection: $\hat{x}_{ij} \approx F(N_{ij})$. A corrupted pixel's value is gone, so the model must use nearby pixels, local structure and context.

Three corruption types

Noise	Definition	Example
Gaussian	$\tilde{x} = x + \varepsilon$, $\varepsilon \sim N(0, \sigma^2 I)$	[0.50,0.80,0.20,0.90] + [0.01,−0.05,0.03,−0.08] = [0.51,0.75,0.23,0.82]
Masking	each component set to 0 with probability q, kept with 1−q	q = 0.4: x = [0.8,0.3,0.9,0.5,0.2] → $\tilde{x}$ = [0.8,0,0.9,0,0.2]
Salt-and-pepper	random pixels set to extreme values (0 = pepper, max = salt)	pixel 130 → 255 (salt), pixel 128 → 0 (pepper)

Masking details: every component is checked independently. With q = 0.4 each has a 40% chance of becoming 0, 60% chance of staying. For 5 values, expected masked = 5 × 0.4 = 2 on average (the actual count varies per draw).

Salt-and-pepper on binary images flips pixels to the opposite extreme (a 0 may become 1 and vice versa). On 8-bit grayscale, values become 0 or 255.

Why pixel neighbors matter

- Neighboring pixels are usually highly correlated (not independent). x_ij denotes the pixel at row i, column j (binary: {0,1}; grayscale: [0,255] or normalized [0,1]).
- Idea: p(x_ij | N_ij), the distribution of the centre pixel given its 3×3 neighborhood.

Worked example A (binary, majority vote)

Neighborhood [[1,1,0],[1,?,1],[0,1,1]] → the 8 neighbors are {1,1,0,1,1,0,1,1}: six 1s, two 0s. P(x=1 | N) = 6/8 = 0.75, P(x=0 | N) = 2/8 = 0.25. Since 0.75 > 0.25, x_ij = 1.

Worked example B (binary)

Neighborhood [[0,0,1],[0,?,0],[0,1,0]]: six 0s, two 1s → P(0) = 0.75 → x_ij = 0.

Worked example C (grayscale, mean)

Neighborhood [[120,125,130],[123,?,127],[122,124,126]]. Sum of neighbors = 120+125+130+123+127+122+124+126 = 997. Mean = 997/8 = 124.625 ≈ 125. So the filled patch has 125 in the centre.

Important caveat from the slide: a real DAE does NOT explicitly average neighbors. It learns such local relationships automatically through convolution filters and training. The averaging is only an intuition.

Convolution link

`Conv2D(32, (3,3), activation='relu', padding='same')` sees the local 3×3 patch around every pixel, so even if the centre pixel is corrupted, the surrounding pixels are still visible. Convolution example from the lecture: 3×3 patch [[3,5,1],[1,0,8],[2,7,2]] with kernel [[1,0,1],[1,0,1],[1,0,1]] = (3+1+2) + (1+8+2) = 6 + 11 = 17 (middle column multiplied by 0).

Why DAE is a regularizer

- Standard AE: x → h → $\hat{x}$, $\hat{x} \approx x$. Copying is a perfect solution.
- DAE: $\tilde{x} \neq x$ generally, so copying the corrupted input does NOT minimize the loss.

- Corruption removes the shortcut of direct copying; the model must infer missing/noisy pixels from context. Result: robust latent features, instead of the identity mapping.

9.2 Sparse autoencoder (SAE), Lecture 14

Problem: if dim(h) < dim(x) the model is forced to compress. If dim(h) > dim(x) it may learn a trivial identity map and overfit. But even with MANY hidden neurons we can force useful learning by applying sparsity to the hidden activations.

Idea: most hidden neurons stay inactive for most inputs; only a few fire to capture important features. Sparsity is imposed with a penalty term added to the loss.

Total loss: L = L_reconstruction + β · L_sparsity

- L_reconstruction: how well the input is reconstructed.
- L_sparsity: penalty when too many neurons become active.
- β: controls the importance of sparsity.
- Two common choices for L_sparsity: KL-divergence penalty (this course's focus) or L1 regularization on activations.

Average activation

Activation of hidden neuron j in layer 2 for input x: a_j(x). For m training samples, the average activation of neuron j is:

ρ̂_j = (1/m) Σ_{i=1..m} a_j(x^(i))

Take neuron j, observe its activation for every training sample, then average. For sparsity we want ρ̂_j close to 0 (rarely active).

KL divergence as sparsity penalty

- ρ = desired average activation (small, e.g. 0.05 or 0.005).
- ρ̂_j = actual average activation of neuron j.
- Goal: ρ̂_j = ρ for every hidden neuron.

KL(ρ ‖ ρ̂_j) = ρ log(ρ / ρ̂_j) + (1 − ρ) log((1 − ρ) / (1 − ρ̂_j))

- First term = "active part": how often the neuron should be active vs how often it actually is.
- Second term = "inactive part": how often it should be inactive vs how often it actually is.

- A neuron has two sides (active and inactive); checking only one would not capture its full behaviour, so both are added.
- Property: if $\hat{\rho}_j = \rho$ then KL = 0. The further $\hat{\rho}_j$ is from ρ, the larger the penalty.
- Total sparsity penalty = $\Sigma_j \mathrm{KL}(\rho \| \hat{\rho}_j)$, added to the reconstruction loss.
- Requirement: KL needs $\hat{\rho}$ strictly between 0 and 1, so the hidden activation is typically sigmoid.
- KL is not symmetric: $\mathrm{KL}(\rho \| \hat{\rho}) \neq \mathrm{KL}(\hat{\rho} \| \rho)$.

Check: perfect match

ρ = $\hat{\rho}$ = 0.05. Active parts 0.05 vs 0.05; inactive parts 0.95 vs 0.95. KL = 0.05 log(0.05/0.05) + 0.95 log(0.95/0.95) = 0.05 log 1 + 0.95 log 1 = 0 (since log 1 = 0). Zero penalty.

Full worked example (lecture): 5 samples, 7 hidden neurons, ρ = 0.005

Activation table:

Sample	h1	h2	h3	h4	h5	h6	h7
x1	0.001	0.30	0.004	0.002	0.006	0.40	0.003
x2	0.002	0.25	0.003	0.001	0.005	0.35	0.004
x3	0.004	0.28	0.006	0.003	0.007	0.45	0.002
x4	0.003	0.32	0.005	0.002	0.008	0.38	0.003
x5	0.005	0.27	0.004	0.001	0.006	0.42	0.005

Step 1: column-wise averages ($\hat{\rho}_j$).

- $\hat{\rho}1$ = (0.001+0.002+0.004+0.003+0.005)/5 = 0.015/5 = 0.003
- $\hat{\rho}2$ = (0.30+0.25+0.28+0.32+0.27)/5 = 1.42/5 = 0.284
- $\hat{\rho}3$ = 0.022/5 = 0.0044
- $\hat{\rho}4$ = 0.009/5 = 0.0018
- $\hat{\rho}5$ = 0.032/5 = 0.0064
- $\hat{\rho}6$ = (0.40+0.35+0.45+0.38+0.42)/5 = 2.00/5 = 0.40
- $\hat{\rho}7$ = 0.017/5 = 0.0034

Step 2: compare with ρ = 0.005. h1, h3, h4, h5, h7 are close (fine). h2 (0.284) and h6 (0.40) are far: much too active.

Step 3: KL for neuron 1 (lecture numbers): KL(0.005 ‖ 0.003) = 0.005 log(0.005/0.003) + 0.995 log(0.995/0.997) = 0.005 × 0.221 + 0.995 × (−0.0008) = 0.00115 − 0.000796 = 0.000354.

Base-of-log warning (important): the lecture values use log base 10 (log10(1.667) = 0.221). Most textbooks and code (e.g. Andrew Ng's sparse AE notes, Keras) use the natural log, which multiplies every KL value by 2.303. Follow whatever base the question states; if unspecified, follow the lecture (base 10).

Step 4: KL values from the lecture table:

Neuron	$\hat{\rho}$	KL(0.005 ‖ $\hat{\rho}$)
h1	0.003	0.0003
h2	0.284	0.13342
h3	0.0044	0.00001
h4	0.0018	0.00083
h5	0.0064	0.00007
h6	0.40	0.2091
h7	0.0034	0.00014

Total sparsity penalty = sum ≈ 0.344, dominated by h2 and h6 (they fire too often). Reading: most neurons are almost always off, so when one does fire it carries relevant information.

You can verify h2: 0.005·log10(0.005/0.284) + 0.995·log10(0.995/0.716) = 0.005(−1.754) + 0.995(0.1429) = −0.00877 + 0.14220 = 0.1334. ✓

Extra practice example (natural log)

$\rho = 0.05$, $\hat{\rho} = 0.2$. KL = 0.05 ln(0.05/0.2) + 0.95 ln(0.95/0.8) = 0.05(−1.3863) + 0.95(0.1719) = −0.0693 + 0.1633 = 0.0940. (In base 10 this is 0.0408.)

How training pushes neurons to sparsity

If a neuron becomes too active, its KL increases → sparsity penalty increases → total loss increases. Backpropagation then adjusts weights and biases so that neuron's activation decreases. Overactive neurons are pushed to fire less.

Sparse AE objective: $J_{sparse}(W, b) = L(x_i, \hat{x}_i) + \beta \Sigma_{j} KL(\rho \| \hat{\rho}_j)$.

Why it regularizes: the network cannot use all hidden neurons freely, so it cannot rebuild the input through a dense trivial representation. It must use a few active neurons efficiently, which encourages meaningful features (even in an overcomplete setting).

9.3 Contractive autoencoder (CAE), Lecture 15

Plain AE says: "from the hidden representation, reconstruct the input well." CAE says: "reconstruct the input well, but keep the hidden representation stable."

Stable means: if two inputs differ slightly, their hidden representations should stay similar. Let x be the original image and $x + \delta$ a slightly changed one (δ = tiny noise, small brightness change, one-pixel variation). A normal AE may give noticeably different hidden vectors for these. CAE tries to make $h(x) \approx h(x + \delta)$.

CAE goal = good reconstruction + stable hidden representation.

Purpose (as stated in the lecture): a regularized AE designed to prevent an overcomplete AE from learning a trivial identity mapping, by adding a Jacobian-based penalty to the reconstruction loss.

Jacobian and penalty

$h = g(x)$, input has n dimensions, hidden layer has k dimensions. The Jacobian $J_x(h)$ is the $k \times n$ matrix of all partial derivatives:

$J_x(h) = [\, \partial h_l / \partial x_j \,]$ with rows $l = 1..k$ (hidden units) and columns $j = 1..n$ (input features).

- Column j holds the derivatives of all hidden units with respect to input feature x_j (the first column is about x_1).
- Entry $\partial h_l/\partial x_j$: if we slightly change x_j, how much does hidden neuron h_l change? It measures sensitivity.
- Large value: h_l is sensitive to x_j (reacts strongly to a small input change). Value near 0: h_l is insensitive to x_j.

Penalty: $\Omega(\theta) = \|J_x(h)\|^2_F = \Sigma_j \Sigma_l (\partial h_l/\partial x_j)^2$

The squared Frobenius norm is the sum of the squares of all elements of the Jacobian. A large Jacobian value means the representation changes a lot even for small input changes. The CAE tries to make these values small.

Total loss: $L_{CAE} = L_{rec}(x, \hat{x}) + \lambda \|J_x(h)\|^2_F$

- L_{rec}: keep enough information to reconstruct well.

- Ω(θ): do not let the hidden representation change too much.
- Ideal value of the second term is 0.

Trade-off and λ

- The model cannot keep every detail AND be completely insensitive. So it learns a balanced representation: preserve important variations, suppress unimportant ones.
- Genuine, meaningful input changes (true dataset patterns) must be noticed, otherwise reconstruction loss rises.
- Changes due to noise, random disturbance or tiny perturbations are unimportant. If the model still reacts to them, derivatives grow and the Jacobian penalty rises, so the penalty discourages strong responses to them.
- λ: how strongly to punish the hidden representation for changing. Higher λ → more stable features (but coarser reconstruction). Lower λ → more detailed reconstruction (less stability).

Worked numerical example (derived; good for exam practice)

For a sigmoid encoder $h = \sigma(Wx + b)$, the derivative is $\partial h_l/\partial x_j = h_l(1 - h_l) W_{lj}$. Therefore

$\|J\|^2_F = \Sigma_l [h_l(1 - h_l)]^2 \cdot \Sigma_j W_{lj}^2$.

Take n = 2 inputs, k = 2 hidden units, W = [[1, 2],[0.5, −1]], b = 0, input x = [0, 0].

- Pre-activations are 0, so h = [0.5, 0.5]; h(1 − h) = 0.25 for both.
- Row sums of squares: row 1 = 1 + 4 = 5; row 2 = 0.25 + 1 = 1.25.
- $\|J\|^2_F = 0.25^2 \times 5 + 0.25^2 \times 1.25 = 0.3125 + 0.078125 = 0.390625$.
- If $L_{rec} = 0.05$ and λ = 0.1: $L_{CAE} = 0.05 + 0.1 \times 0.390625 = 0.0890625$.

Note how scaling W up increases the penalty (quadratically), so the CAE implicitly pushes weights/activations toward saturation or smaller magnitude: the representation becomes flatter around the data.

9.4 Comparison of regularized AEs

	Denoising	Sparse	Contractive
What is regularized	Input (corrupted)	Hidden activations	Sensitivity of h to x

	Denoising	Sparse	Contractive
Mechanism	Corrupt x → x̃, reconstruct clean x	Penalty KL(ρ ‖ ρ̂) (or L1) on activations	Jacobian Frobenius-norm penalty
Hyperparameter	noise level (σ or q)	β, ρ	λ
Learns	Robust features, uses neighbor context	Few active neurons, meaningful features	Stable, noise-insensitive features
Prevents	Trivial identity copy	Dense trivial representation	Identity mapping / over-sensitivity

---

10. Full numerical example (Lecture 16): 5 → 4 → 3 → 4 → 5

Architecture: input 5, encoder hidden 4, latent 3, decoder hidden 4, output 5. ReLU in hidden layers, linear at the output (inputs continuous), MSE loss. Each layer has a single shared bias value (0.01 per layer).

Input: x = [0.8, 0.4, 0.6, 0.2, 0.5] Formula per layer: output = activation(input · W + b). Column j of W gives neuron j.

Step 1: Encoder, input → hidden 1 (5 → 4)

W1 (5×4) = [[0.2,0.1,0.4,0.3],[0.5,0.3,0.2,0.1],[0.4,0.2,0.1,0.5],[0.3,0.6,0.2,0.4], [0.1,0.4,0.3,0.2]], b1 = 0.01.

- h11 = 0.8(0.2)+0.4(0.5)+0.6(0.4)+0.2(0.3)+0.5(0.1)+0.01 = 0.16+0.20+0.24+0.06+0.05+0.01 = 0.72 → ReLU = 0.72
- h12 = 0.8(0.1)+0.4(0.3)+0.6(0.2)+0.2(0.6)+0.5(0.4)+0.01 = 0.08+0.12+0.12+0.12+0.20+0.01 = 0.65
- h13 = 0.8(0.4)+0.4(0.2)+0.6(0.1)+0.2(0.2)+0.5(0.3)+0.01 = 0.32+0.08+0.06+0.04+0.15+0.01 = 0.66
- h14 = 0.8(0.3)+0.4(0.1)+0.6(0.5)+0.2(0.4)+0.5(0.2)+0.01 = 0.24+0.04+0.30+0.08+0.10+0.01 = 0.77

h1 = [0.72, 0.65, 0.66, 0.77]. All positive, so ReLU leaves them unchanged.

Step 2: Encoder, hidden 1 → latent (4 → 3)

W2 (4×3) = [[0.3,0.2,0.4],[0.5,0.1,0.3],[0.2,0.4,0.1],[0.1,0.3,0.5]], b2 = 0.01.

- z1 = 0.72(0.3)+0.65(0.5)+0.66(0.2)+0.77(0.1)+0.01 = 0.216+0.325+0.132+0.077+0.01 = 0.760
- z2 = 0.72(0.2)+0.65(0.1)+0.66(0.4)+0.77(0.3)+0.01 = 0.144+0.065+0.264+0.231+0.01 = 0.714
- z3 = 0.72(0.4)+0.65(0.3)+0.66(0.1)+0.77(0.5)+0.01 = 0.288+0.195+0.066+0.385+0.01 = 0.944

Latent z = [0.760, 0.714, 0.944]. So 5 values are compressed into 3: [0.8,0.4,0.6,0.2,0.5] → [0.760,0.714,0.944].

Step 3: Decoder, latent → decoder hidden (3 → 4)

W3 (3×4) = [[0.4,0.1,0.3,0.2],[0.2,0.5,0.1,0.4],[0.3,0.2,0.4,0.1]], b3 = 0.01.

- h21 = 0.760(0.4)+0.714(0.2)+0.944(0.3)+0.01 = 0.304+0.1428+0.2832+0.01 = 0.740
- h22 = 0.760(0.1)+0.714(0.5)+0.944(0.2)+0.01 = 0.076+0.357+0.1888+0.01 = 0.6318
- h23 = 0.760(0.3)+0.714(0.1)+0.944(0.4)+0.01 = 0.228+0.0714+0.3776+0.01 = 0.687
- h24 = 0.760(0.2)+0.714(0.4)+0.944(0.1)+0.01 = 0.152+0.2856+0.0944+0.01 = 0.542

h2 = [0.740, 0.6318, 0.687, 0.542] (expansion 3 → 4).

Step 4: Decoder, hidden → output (4 → 5), linear activation f(a) = a

W4 (4×5) = [[0.3,0.2,0.4,0.1,0.2],[0.1,0.5,0.2,0.3,0.4],[0.4,0.1,0.3,0.2,0.1],[0.2,0.3,0.1,0.5,0.3]], b4 = 0.01.

- $\hat{x}1$ = 0.740(0.3)+0.6318(0.1)+0.687(0.4)+0.542(0.2)+0.01 = 0.222+0.06318+0.2748+0.1084+0.01 = 0.67838
- $\hat{x}2$ = 0.740(0.2)+0.6318(0.5)+0.687(0.1)+0.542(0.3)+0.01 = 0.148+0.3159+0.0687+0.1626+0.01 = 0.7052
- $\hat{x}3$ = 0.740(0.4)+0.6318(0.2)+0.687(0.3)+0.542(0.1)+0.01 = 0.296+0.12636+0.2061+0.0542+0.01 = 0.69266
- $\hat{x}4$ = 0.740(0.1)+0.6318(0.3)+0.687(0.2)+0.542(0.5)+0.01 = 0.074+0.18954+0.1374+0.271+0.01 = 0.68194
- $\hat{x}5$ = 0.740(0.2)+0.6318(0.4)+0.687(0.1)+0.542(0.3)+0.01 = 0.148+0.25272+0.0687+0.1626+0.01 = 0.64202

$\hat{x}$ = [0.67838, 0.7052, 0.69266, 0.68194, 0.64202].

Step 5: Reconstruction loss (MSE)

i	x_i	$\hat{x}$_i	error	squared
1	0.8	0.67838	0.12162	0.01479
2	0.4	0.7052	−0.3052	0.09315
3	0.6	0.69266	−0.09266	0.00859
4	0.2	0.68194	−0.48194	0.23227
5	0.5	0.64202	−0.14202	0.02017

Sum = 0.36896; MSE = 0.36896 / 5 = 0.07379.

Interpretation: the (untrained, random) weights give a reconstruction that is almost a flat 0.65–0.7 vector. Training adjusts all weights and biases by backpropagation to push $\hat{x}$ toward x, reducing MSE. After training: $x \cdot W^* \rightarrow h^* \rightarrow h^* W^{*T} \rightarrow \hat{x}$ (tied-weights form).

I re-computed every step above and the slide arithmetic is correct. Exam tips: always show (a) which column of W you use, (b) the bias added once per neuron, (c) the ReLU/linear step, (d) the final MSE divided by the number of features.

Why linear output + MSE here

Inputs are continuous, so the output must be continuous: f(a) = a returns the computed value directly. If inputs were in [0,1] / binary, use sigmoid + BCE instead.

---

11. Limitations of standard autoencoders (Lecture 16)

1. Low reconstruction error does not mean meaningful features. $L_{AE} = \|x - \hat{x}\|^2$ can be small while the latent variables fail to represent semantic factors, e.g. in handwritten digits the model may reconstruct a "2" well, but the latent code may not clearly encode whether the digit is 2 or 9, whether the stroke is thick or thin, or whether the digit is slanted.
2. The compressed information is not guaranteed to be useful for downstream tasks such as classification, prediction, clustering or anomaly detection, even when reconstruction error is low.

3. Because the output is generated from a compressed latent code, the decoder only produces an approximation of the input; the model cannot generate new samples (the latent space is not organized for sampling; random points in it need not decode to anything sensible).
4. Choosing the latent dimension is critical:
   - Too small: excessive compression → information loss → poor (blurry, incomplete, distorted) reconstruction.
   - Too large: the model may simply memorize the input → low reconstruction error but weak feature learning (no compact, useful representation).

Solution direction: Variational Autoencoders (VAEs) address these limitations by regularizing the latent space. This is done by adding a KL-divergence regularization term to the loss: Loss = Reconstruction + KL. (Next week's topic; the same KL idea you saw in sparse AEs, but now between the latent distribution and a prior.)

---

12. Formula sheet

Item	Formula
Encoder	$h = g(Wx + b)$ or $H = g(X\,W_e)$
Decoder	$\hat{x} = f(W'h + c)$ or $\hat{X} = f(H\,W_d)$
Tied weights	$W_d = W_e^T$
Shapes	X: m×n, W_e: n×k, H: m×k, W_d: k×n
MSE (sample)	$(1/d)\,\Sigma\,(x_i - \hat{x}_i)^2$; per-sample loss $\|x - \hat{x}\|^2$
BCE	$-\,\Sigma\,[x \log \hat{x} + (1-x) \log(1-\hat{x})]$
Regularized	$f(W) = \min \|X - XWW^T\|^2 + \lambda\|W\|^2$
Gaussian noise	$\tilde{x} = x + \varepsilon,\ \varepsilon \sim N(0, \sigma^2 I)$
Masking	$\tilde{x}_ij = 0$ w.p. q, x_ij w.p. 1−q; expected masked = q·d
Avg activation	$\hat{\rho}_j = (1/m)\,\Sigma\, a_j(x^{(i)})$
KL sparsity	$\rho \log(\rho/\hat{\rho}) + (1-\rho) \log((1-\rho)/(1-\hat{\rho}))$
Sparse loss	$L_rec + \beta\,\Sigma\, KL(\rho \,\|\, \hat{\rho}_j)$

Item	Formula
CAE penalty	$\|J\|^2_F = \Sigma\Sigma (\partial h_l/\partial x_j)^2$
CAE loss	$L_{rec} + \lambda \|J\|^2_F$
Transposed conv size	$(n-1)s + k - 2p$ (stride 1, p = 0: $n + k - 1$)
Upsampling size	input × S

Quick decision table:

Situation	Choose
Binary or [0,1] data	sigmoid output + BCE
Real-valued data	linear output + MSE
Images	CNN AE (conv + pool, upsample/transposed conv)
Overcomplete, want robustness to noise	Denoising AE
Overcomplete, want few active units	Sparse AE
Want stable representation for small input changes	Contractive AE
Want to generate new samples	VAE

13. Common exam traps

- DAE reconstructs the CLEAN x, not the noisy $\tilde{x}$.
- Number of output neurons = number of input features.
- Tied weights: $W_d = W_e^T$, so parameters halve; shapes must be reversed.
- Masking noise: "approximately q·d values are masked on average", not exactly.
- Sparse AE penalizes the AVERAGE activation over the training set, not a single sample.
- KL = 0 only when $\hat{\rho} = \rho$; it grows both when $\hat{\rho}$ is too large and too small.
- Contractive AE penalizes the derivative of h with respect to x (the Jacobian), not the weights directly (that would be weight decay).
- Higher λ in CAE → more stable, less detailed. Lower λ → more detailed, less stable.

- Upsampling has no learnable parameters; transposed convolution does.
- Transposed convolution → checkerboard artifacts; upsampling + convolution avoids them.
- Low reconstruction error ≠ good features.

---

14. Practice MCQs (answers and explanations follow each question)

Q1. In an autoencoder, the training target is: A) A class label B) The input itself C) A noisy version of the input D) A random vector **Answer: B.** The special idea of AEs is that the input is used as the target, so no labels are needed.

Q2. An autoencoder with architecture 8 → 3 → 8 is: A) Overcomplete B) Undercomplete C) Sparse D) Contractive **Answer: B.** dim(h) = 3 < dim(x) = 8.

Q3. Which architecture is overcomplete? A) 6 → 3 → 6 B) 100 → 75 → 25 → 75 → 100 C) 4 → 6 → 4 D) 784 → 32 → 784 **Answer: C.** Hidden size 6 ⩾ input size 4, so it can learn a trivial copy.

Q4. For input with 100 features, the output layer of the AE has: A) 25 neurons B) 75 neurons C) 100 neurons D) Any number **Answer: C.** The output must reconstruct the input, so it has as many neurons as input features.

Q5. X is 1000 × 6 and the latent size is 3. The shapes of W_e, H and W_d are: A) 6×3, 1000×3, 3×6 B) 3×6, 1000×3, 6×3 C) 6×3, 1000×6, 3×6 D) 1000×3, 6×3, 3×6 **Answer: A.** H = X W_e (1000×6 times 6×3 = 1000×3); $\hat{X}$ = H W_d needs W_d of 3×6.

Q6. With tied weights for a 6 → 3 → 6 AE (ignore biases), the number of learnable weights is: A) 36 B) 18 C) 9 D) 6 **Answer: B.** Only W_e (6×3 = 18) is learned; $W_d = W_e^T$.

Q7. Inputs are binary (0/1). The most suitable decoder activation and loss are: A) Linear + MSE B) ReLU + MSE C) Sigmoid + BCE D) Linear + BCE **Answer: C.** Output must lie in (0,1) and be read as a probability.

Q8. Inputs are standardized real values. The decoder should use: A) Sigmoid + BCE B) Linear + MSE C) Softmax + BCE D) Sigmoid + MSE **Answer: B.** Outputs must be able to take any real value.

Q9. x = [1, 0], $\hat{x}$ = [0.8, 0.2]. The MSE is: A) 0.04 B) 0.08 C) 0.4 D) 0.2 **Answer: A.** $((0.2)^2 + (0.2)^2)/2 = 0.08/2 = 0.04$.

Q10. BCE for one feature with x = 1 and $\hat{x}$ = 0.5 (natural log) is: A) 0.5 B) 0.693 C) 0 D) 1.386 **Answer: B.** $-\ln(0.5) = 0.693$.

Q11. A denoising autoencoder is trained so that its output is close to: A) The noisy input $\tilde{x}$ B) The clean input x C) Random noise D) The latent code **Answer: B.** Input is corrupted, target is the original clean image.

Q12. Masking noise with q = 0.4 is applied independently to 10 features. The expected number of features set to zero is: A) 6 B) 4 C) 2.5 D) Exactly 4 every time **Answer: B.** Expected = q × d = 4 on average; the actual count varies (so D is wrong).

Q13. In salt-and-pepper noise on an 8-bit grayscale image, corrupted pixels become: A) Gaussian values B) 0 or 255 C) The neighborhood mean D) Random values in (0,1) **Answer: B.** Pepper = 0 (black), salt = 255 (white).

Q14. Centre pixel of a binary 3×3 patch is missing. The 8 neighbors contain five 1s and three 0s. By the neighborhood estimate, P(centre = 1) and the inferred value are: A) 0.375, 0 B) 0.625, 1 C) 0.5, 1 D) 0.625, 0 **Answer: B.** 5/8 = 0.625 > 3/8, so infer 1.

Q15. Grayscale neighbors are {10, 20, 30, 40, 50, 60, 70, 80}. The mean-based estimate of the missing centre pixel is: A) 40 B) 45 C) 50 D) 360 **Answer: B.** Sum = 360, 360/8 = 45.

Q16. Why is a denoising AE a regularized variant? A) It adds a KL penalty B) It removes the shortcut of copying the input C) It uses fewer layers D) It uses linear activations **Answer: B.** Because $\tilde{x} \neq x$, copying does not minimize the loss; the model must use context.

Q17. Why does a 3×3 Conv2D layer help a denoising AE? A) It removes the need for a loss function B) It sees the local neighborhood of each pixel C) It increases the latent size D) It makes the model linear **Answer: B.** Even if the centre pixel is corrupted, the filter still sees the surrounding pixels.

Q18. In a sparse AE, $\hat{\rho}_j$ is: A) The activation of neuron j for one sample B) The average activation of neuron j over the training set C) The desired sparsity D) The weight of neuron j **Answer: B.** $\hat{\rho}_j = (1/m) \Sigma\, a_j(x^{(i)})$.

Q19. The sparsity penalty $KL(\rho \,\|\, \hat{\rho}_j)$ equals zero when: A) $\hat{\rho}_j = 0$ B) $\hat{\rho}_j = 1$ C) $\hat{\rho}_j = \rho$ D) $\rho = 1$ **Answer: C.**

Q20. A neuron has activations [0.2, 0.4, 0.6, 0.8] over four samples. Its $\hat{\rho}$ is: A) 0.4 B) 0.5 C) 0.6 D) 2.0 **Answer: B.** Sum = 2.0, divided by 4 = 0.5.

Q21. With $\rho = 0.005$, which neuron gets the largest sparsity penalty? A) $\hat{\rho} = 0.003$ B) $\hat{\rho} = 0.0044$ C) $\hat{\rho} = 0.40$ D) $\hat{\rho} = 0.0034$ **Answer: C.** It is by far the furthest from ρ (fires far too often).

Q22. Total loss of a sparse autoencoder is: A) $L_{rec} - \beta \Sigma KL$ B) $L_{rec} + \beta \Sigma\, KL(\rho \,\|\, \hat{\rho}_j)$ C) Σ KL only D) $L_{rec} \times \beta$ **Answer: B.**

Q23. KL-divergence sparsity typically assumes the hidden activation is: A) Linear B) Sigmoid, so that $\hat{\rho}$ lies in (0,1) C) Softmax D) Any, it does not matter **Answer: B.** The logs in KL require $\hat{\rho}$ strictly between 0 and 1.

Q24. In a contractive AE, the regularizer is: A) The L2 norm of the weights only B) The squared Frobenius norm of the Jacobian of h with respect to x C) KL divergence of activations D) The noise variance **Answer: B.**

Q25. In CAE, a large $\partial h_1/\partial x_1$ means: A) h_1 is insensitive to x_1 B) h_1 is sensitive to x_1 C) x_1 is zero D) The loss is minimized **Answer: B.**

Q26. Increasing λ in a CAE leads to: A) More detailed reconstruction B) More stable (less sensitive) features C) A larger latent space D) No change **Answer: B.** Higher λ punishes sensitivity more, giving more stable features at the cost of reconstruction detail.

Q27. A CAE aims to make: A) $h(x) \approx h(x + \delta)$ for small δ B) $\hat{x} = 0$ C) h large D) $h(x)$ very different from $h(x + \delta)$ **Answer: A.**

Q28. Which statement about upsampling (nearest neighbor/bed of nails) is true? A) It has learnable parameters B) It has no learnable parameters C) It decreases spatial size D) It always causes checkerboard artifacts **Answer: B.**

Q29. A 2×2 input is transposed-convolved with a 2×2 kernel (stride 1, no padding). The output size is: A) 2×2 B) 3×3 C) 4×4 D) 1×1 **Answer: B.** $(2 + 2 - 1) = 3$ in each dimension.

Q30. Checkerboard artifacts arise from: A) Random noise in data B) Uneven overlap of the kernel in transposed convolution C) Too many filters D) ReLU **Answer: B.** It is a systematic artifact of the operation; upsampling followed by convolution avoids it.

Q31. In the lecture's CNN AE, the 28×28 input becomes 7×7 at the latent layer because: A) Conv with stride 4 B) Two MaxPooling2D(2,2) layers C) Two UpSampling layers D) Padding='same' **Answer: B.** 28 → 14 → 7. padding='same' keeps size.

Q32. The last layer of the CNN AE uses sigmoid because: A) Pixels are normalized to [0,1] B) It is faster C) ReLU is not allowed D) Latent is binary **Answer: A.**

Q33. Compared with a dense autoencoder, a CNN autoencoder on images: A) Flattens the image first B) Preserves local spatial structure (edges, textures) C) Cannot be trained D) Has no decoder **Answer: B.**

Q34. A key limitation of standard autoencoders is: A) They need labels B) Low reconstruction error does not guarantee meaningful features C) They cannot be trained with backpropagation D) They can only process text **Answer: B.**

Q35. If the latent dimension is too large, a plain AE may: A) Produce blurry output B) Memorize the input with low reconstruction error but weak features C) Fail to reconstruct D) Become variational **Answer: B.**

Q36. Which model addresses the limitations (regularizes the latent space) with a KL term in the loss? A) Shallow AE B) Denoising AE C) Variational AE D) Deep AE **Answer: C.**

Q37. In the numerical example, h_1 = [0.72, 0.65, 0.66, 0.77] → z uses W2 with b2 = 0.01. z_1 is computed from which column of W2? A) First column (0.3, 0.5, 0.2, 0.1) B) First row C) Last column D) Diagonal **Answer: A.** Column j of W gives neuron j: 0.72(0.3)+0.65(0.5)+0.66(0.2)+0.77(0.1)+0.01 = 0.760.

Q38. In the numerical example the final MSE is: A) 0.7379 B) 0.07379 C) 0.36896 D) 0.0148 **Answer: B.** Sum of squares 0.36896 divided by 5 features.

Q39. The output activation in the numerical example is linear because: A) Inputs are continuous B) It is cheaper C) ReLU cannot be used in the decoder D) The latent is 3-D **Answer: A.**

Q40. Which AE can an organized latent space let us sample new data from? A) Standard AE B) Sparse AE C) VAE D) Contractive AE **Answer: C.** Standard AEs only approximate inputs; they are not built to generate new samples.

---

15. Numerical practice problems with full solutions

Problem 1: forward pass and MSE

x = [1, 2]. Encoder W1 = [[0.5, −1],[0.25, 0.5]] (rows = inputs, columns = neurons), b = 0.1, ReLU. Decoder W2 = identity, b = 0, linear. Find h and MSE.

Solution:

- h_1 = 1(0.5) + 2(0.25) + 0.1 = 1.1 → ReLU = 1.1
- h_2 = 1(−1) + 2(0.5) + 0.1 = 0.1 → ReLU = 0.1
- $\hat{x}$ = [1.1, 0.1] (identity, linear)
- MSE = $((1 - 1.1)^2 + (2 - 0.1)^2)/2 = (0.01 + 3.61)/2 = 1.81$.

Problem 2: KL divergence (natural log)

$\rho = 0.1$ and $\hat{\rho} = 0.5$. Find $KL(\rho \| \hat{\rho})$.

Solution: 0.1 ln(0.1/0.5) + 0.9 ln(0.9/0.5) = 0.1(−1.6094) + 0.9(0.5878) = −0.1609 + 0.5290 = 0.368. For $\hat{\rho} = 0.1$ it would be exactly 0.

Problem 3: transposed convolution

Input [[2,0],[1,3]], kernel [[1,2],[0,1]]. Find the 3×3 output.

Solution:

- 2 × K = [[2,4],[0,2]] placed at rows 0–1, cols 0–1.
- 0 × K = zeros (input position (0,1)).
- 1 × K = [[1,2],[0,1]] placed at rows 1–2, cols 0–1.
- 3 × K = [[3,6],[0,3]] placed at rows 1–2, cols 1–2.
- Add overlaps: result = [[2,4,0],[1,7,6],[0,1,3]]. Check: the sum of all outputs = (sum of input) × (sum of kernel) = 6 × 4 = 24, and 2+4+0+1+7+6+0+1+3 = 24. ✓ (Centre cell 7 = 2 + 2 + 3 from three overlapping placements.)

Problem 4: contractive penalty

One sigmoid hidden unit, weights W = [2, 0] on two inputs, with h = 0.5 at the point of interest. λ = 0.2, L_rec = 0.1. Find L_CAE.

Solution: h(1 − h) = 0.25. Jacobian = [0.25·2, 0.25·0] = [0.5, 0]. $\|J\|^2_F = 0.25$. L_CAE = 0.1 + 0.2(0.25) = 0.15.

Problem 5: sparse average activation

A neuron has activations [0.1, 0.3, 0.2, 0.2] over 4 samples and ρ = 0.2. Is the penalty zero?

Solution: $\hat{\rho} = 0.8/4 = 0.2 = \rho$, so KL = 0. Note: the average matches even though individual activations differ; KL only looks at the average.

Problem 6: parameter count of the deep autoencoder (with biases)

784 → 128 → 64 → 32 → 64 → 128 → 784.

- 784×128 + 128 = 100,480
- 128×64 + 64 = 8,256
- 64×32 + 32 = 2,080
- 32×64 + 64 = 2,112
- 64×128 + 128 = 8,320

- 128×784 + 784 = 101,136 Total = 222,384 parameters.

Problem 7: parameter count of the CNN autoencoder

Conv2D parameters = k×k×(input channels)×(filters) + filters. Pooling and upsampling have none.

- enc_conv1: 3×3×1×64 + 64 = 640
- enc_conv2: 3×3×64×128 + 128 = 73,856
- latent: 3×3×128×256 + 256 = 295,168
- dec_conv1: 3×3×256×128 + 128 = 295,040
- dec_conv2: 3×3×128×64 + 64 = 73,792
- output: 3×3×64×1 + 1 = 577 Total = 739,073 parameters.

Problem 8: masking probability

Five features with q = 0.3. Probability that none is masked? Each is kept with 0.7, independently: $0.7^5 = 0.168$. Probability that all are masked: $0.3^5 = 0.0024$. Expected masked: 5 × 0.3 = 1.5.

---

16. Likely descriptive / short-answer questions (practice list)

1. Explain the two objectives of an autoencoder with a labelled diagram (encoder compression, decoder reconstruction).
2. Differentiate undercomplete and overcomplete AEs. Why is the overcomplete case a problem?
3. Explain tied weights and state their advantage.
4. When do you use MSE vs BCE? State the matching output activations.
5. Describe three noise types used in denoising AEs with a numerical example each.
6. Explain how a denoising AE learns neighboring-pixel dependencies and why this regularizes the model.
7. Define sparsity in an AE; derive/explain the KL sparsity penalty; solve a KL numerical.
8. Explain the contractive AE: Jacobian, Frobenius norm, role of λ, meaning of the trade-off.
9. Compare nearest-neighbor upsampling, bed-of-nails, transposed convolution; explain checkerboard artifacts and the upsampling + convolution fix.

10. Compare a DNN AE and a CNN AE on images.
11. List the limitations of standard autoencoders and explain why VAEs are needed.
12. Do the full 5 → 4 → 3 → 4 → 5 forward pass and MSE (be ready with your own weights).

17. Last-minute revision checklist

- Can you draw AE architecture and label encoder, latent code, decoder?
- Do you remember: sigmoid + BCE for binary/[0,1], linear + MSE for real values?
- Can you do a forward pass by hand, using columns of W for each neuron?
- Can you compute $\hat{\rho}$ and KL for a given table?
- Can you compute transposed-conv outputs by overlap-and-add, and upsample + conv by sliding windows?
- Can you contrast DAE (corrupt input), SAE (penalize activation), CAE (penalize Jacobian)?
- Can you state the 4 limitations and the VAE fix?

PART B – DEEP DIVE: INTUITION, DERIVATIONS, CODE, EXTRA NUMERICALS AND QUESTION BANK

Part A (sections 0–17 above) is the lecture-aligned summary. Part B explains why each idea works, derives the maths step by step, gives complete code, and adds many more solved numericals and questions. Read Part A for what is on the slides; use Part B to actually understand it and to handle questions that twist the slide examples.

B1. The big idea explained slowly

B1.1 What problem are we solving?

Real data is high-dimensional (a 28×28 image has 784 numbers), but the interesting information in it usually lives in far fewer degrees of freedom. A handwritten "2" is described by a few things: which digit, how thick the stroke is, how slanted it is, how large

it is. Those few factors generate all 784 pixel values. If we could find those few factors automatically, we would have a compact, meaningful description of the data (a representation).

Supervised learning gets its guidance from labels. When no labels exist, we need a different training signal. The autoencoder's trick: use the input itself as the label. Force the network to send the input through a narrow channel and then rebuild it. For the rebuild to succeed, the narrow channel must carry the information that matters. Nothing else gives it a reason to keep anything.

Analogy: you read a chapter, write a 5-line summary, hand only the summary to a friend, and the friend tries to rewrite the chapter. If the friend's rewrite is close, your summary kept the essential content. The summary is the latent code, you are the encoder, the friend is the decoder, and "how close the rewrite is" is the reconstruction loss.

B1.2 Why a bottleneck is necessary

If the middle layer is as wide as (or wider than) the input, the network can pass the input straight through (identity). Reconstruction error becomes zero but nothing has been learned. The bottleneck (dim(h) < dim(x)) removes that shortcut. Regularization (denoising, sparsity, contractivity) removes the shortcut in other ways when the middle layer is wide.

Way of preventing the identity shortcut	Mechanism
Undercomplete	Narrow latent layer; cannot hold everything
Denoising	Input is damaged; copying it reproduces the damage, which is wrong
Sparse	Only a few hidden units may be active; cannot spread input over all units
Contractive	Hidden code may not react to tiny input changes; cannot pass through every detail

B1.3 What "representation" means

The latent vector h is a new coordinate system for the same data. Good representations make later tasks easier (classification, clustering, anomaly detection, visualization). The lecture's warning in the limitations section is exactly that a plain AE does NOT guarantee this: low reconstruction error is necessary, not sufficient, for a useful representation.

B1.4 Encoder and decoder as functions

- Encoder: $h = g(Wx + b)$. Compression. Typically each layer has fewer neurons than the last.
- Decoder: $\hat{x} = f(W'h + c)$. Expansion. Typically a mirror image of the encoder.
- Whole network: $\hat{x} = f(W'\, g(Wx + b) + c)$. It is trained end to end as ONE network by backpropagation, with loss $L(x, \hat{x})$.
- After training, you can use only the encoder as a feature extractor, or only the decoder as a generator-like module (with caveats, see limitations).

B1.5 The three levels of understanding for the exam

1. Structure: input → encoder → latent → decoder → output; output size = input size.
2. Objective: minimize reconstruction loss; MSE for real data with linear output, BCE for binary data with sigmoid output.
3. Failure modes and fixes: identity mapping (fix: regularize), no generation and no guarantee of useful features (fix: VAE).

B2. Linear autoencoder and PCA (a classic exam connection)

If the encoder and decoder are linear (no nonlinear activation) and the loss is MSE, the optimal autoencoder learns the same subspace as the top-k principal components of the data (PCA). The latent code is a rotated/scaled version of the PCA projection. Nonlinear AEs (ReLU, sigmoid, tanh hidden layers) are strictly more expressive: they can model curved structure (manifolds) that PCA cannot.

Worked example: perfect compression of data on a line

Data points lie on $y = 2x$: (1,2), (2,4), (3,6). Use a 1-D latent code. Choose encoder weight vector $w = [1, 2]/\sqrt{5}$ (unit vector along the line) and tied decoder (w^T).

For x = (1,2): z = x·w = (1·1 + 2·2)/√5 = 5/√5 = √5 ≈ 2.236. Reconstruction x̂ = z·w = √5 · [1,2]/√5 = [1, 2]. Error 0. For x = (3,6): z = (3 + 12)/√5 = 15/√5 = 3√5. x̂ = 3√5 · [1,2]/√5 = [3, 6]. Error 0.

Two numbers were compressed into one with zero loss because the data really has only one degree of freedom. That is what the bottleneck is hunting for.

Worked example: a bottleneck that is too small

Now use data that is NOT on a line: x = (3, 4). With the same w: z = (3 + 8)/√5 = 11/√5 = 4.919. x̂ = (11/√5)·[1,2]/√5 = (11/5)·[1,2] = [2.2, 4.4]. Error = [0.8, −0.4]; squared error = 0.64 + 0.16 = 0.80; MSE = 0.40. The information perpendicular to the line was discarded. This is the "latent too small → information loss" case.

B3. Backpropagation through an autoencoder (one full hand-computed step)

This is the part the slides leave implicit ("the autoencoder tries to adjust all weights"). Here it is concretely.

Setup (linear AE so the algebra is clear): input x = [1, 2]. Encoder weights w = [0.5, 0.25] giving a 1-D latent h = w·x. Decoder weights v = [0.4, 0.8] giving x̂ = h·v. No biases. Loss L = Σ (x̂_i − x_i)². Learning rate η = 0.1.

Forward pass

- h = 0.5·1 + 0.25·2 = 0.5 + 0.5 = 1.0
- x̂ = 1.0 · [0.4, 0.8] = [0.4, 0.8]
- error e = x̂ − x = [0.4 − 1, 0.8 − 2] = [−0.6, −1.2]
- L = 0.36 + 1.44 = 1.80

Backward pass (chain rule)

- ∂L/∂x̂ = 2e = [−1.2, −2.4]
- ∂L/∂v_i = (∂L/∂x̂_i) · h = [−1.2, −2.4] · 1.0 = [−1.2, −2.4]
- ∂L/∂h = Σ (∂L/∂x̂_i) v_i = (−1.2)(0.4) + (−2.4)(0.8) = −0.48 − 1.92 = −2.40
- ∂L/∂w_j = (∂L/∂h) · x_j = −2.40 · [1, 2] = [−2.4, −4.8]

Update (gradient descent: parameter ← parameter − η · gradient)

- v_new = [0.4 + 0.12, 0.8 + 0.24] = [0.52, 1.04]
- w_new = [0.5 + 0.24, 0.25 + 0.48] = [0.74, 0.73]

Forward pass again

- h = 0.74·1 + 0.73·2 = 0.74 + 1.46 = 2.20
- $\hat{x}$ = 2.20 · [0.52, 1.04] = [1.144, 2.288]
- e = [0.144, 0.288], L = 0.020736 + 0.082944 = 0.1037

Loss fell from 1.80 to 0.104 in one step. (A step this large is lucky; with big learning rates the loss can overshoot, which is why η is tuned.) This is exactly what "adjust all weights and biases so the reconstruction gets closer to the input" means.

Rule worth memorizing: gradients flow from the output back through the decoder, then through the latent code, then into the encoder. Encoder weights are updated using information about how bad the reconstruction was.

With tied weights ($W_d = W_e^T$) the same matrix appears in both places, so its gradient is the sum of the contribution from the encoder use and the contribution from the decoder use.

B4. Activation functions and their derivatives (needed for backprop questions)

Function	Formula	Output range	Derivative	Used for
ReLU	max(0, x)	[0, ∞)	1 if x > 0 else 0	hidden layers
Sigmoid	1/(1 + e^(−x))	(0, 1)	σ(x)(1 − σ(x)); max 0.25 at x = 0	binary/[0,1] output; sparse and contractive hidden units
Tanh	(e^x − e^(−x))/(e^x + e^(−x))	(−1, 1)	1 − tanh²(x)	hidden layers
Linear	x	(−∞, ∞)	1	real-valued output

Quick values: σ(0) = 0.5; σ(2) = 0.881; σ(−2) = 0.119; ReLU(−3) = 0; ReLU(0.77) = 0.77.

Note for the numerical example in Lecture 16: all pre-activations are positive, so ReLU acts like the identity there. If any pre-activation had been negative, that neuron would output exactly 0 (a "dead" neuron for that input).

Why sigmoid + BCE is a natural pair

For sigmoid output $\hat{x} = \sigma(a)$ with BCE loss, the gradient of the loss with respect to the pre-activation a simplifies to $(\hat{x} - x)$. This clean gradient avoids the slow learning that sigmoid + MSE can cause when the output saturates. It is one reason binary data uses BCE. Similarly, linear output + MSE gives a gradient proportional to $(\hat{x} - x)$.

B5. Another complete forward pass (practice, binary data, BCE)

The lecture example is real-valued with MSE. Examiners can switch to binary data. Work through this one.

Architecture 3 → 2 → 3. Input x = [1, 0, 1]. Biases are 0. Hidden activation ReLU, output activation sigmoid, loss BCE (natural log).

Encoder weights W1 (3×2), rows are inputs, columns are hidden neurons: [[0.5, −0.5],[0.2, 0.1],[−0.3, 0.8]]

Hidden neuron 1 (column 1): 1(0.5) + 0(0.2) + 1(−0.3) = 0.2 → ReLU = 0.2 Hidden neuron 2 (column 2): 1(−0.5) + 0(0.1) + 1(0.8) = 0.3 → ReLU = 0.3 h = [0.2, 0.3]

Decoder weights W2 (2×3): [[1, −1, 0.5],[0.5, 2, −1]]

Output pre-activations:

- a1 = 0.2(1) + 0.3(0.5) = 0.35
- a2 = 0.2(−1) + 0.3(2) = 0.40
- a3 = 0.2(0.5) + 0.3(−1) = −0.20

Sigmoid:

- σ(0.35) = 1/(1 + e^(−0.35)) = 1/(1 + 0.7047) = 0.5866
- σ(0.40) = 1/(1 + 0.6703) = 0.5987
- σ(−0.20) = 1/(1 + 1.2214) = 0.4502

$\hat{x}$ = [0.5866, 0.5987, 0.4502]

BCE terms (x = [1, 0, 1]):

- feature 1 (x = 1): −ln(0.5866) = 0.534
- feature 2 (x = 0): −ln(1 − 0.5987) = −ln(0.4013) = 0.913
- feature 3 (x = 1): −ln(0.4502) = 0.798

Sample loss = 0.534 + 0.913 + 0.798 = 2.245 (about 0.748 averaged over the 3 features).

Interpretation: with untrained weights the output hovers around 0.5, so each feature costs about −ln(0.5) = 0.69; training pushes $\hat{x}$ toward [1, 0, 1].

B6. More small numericals (hand-solvable in an exam)

B6.1 Linear AE with tied weights where information is lost

x = [2, 1, 0], W_e = [[1,0],[0,1],[1,1]] (3×2), W_d = W_e^T = [[1,0,1],[0,1,1]].

- h = x W_e = [2·1 + 1·0 + 0·1, 2·0 + 1·1 + 0·1] = [2, 1]
- $\hat{x}$ = h W_d = [2·1 + 1·0, 2·0 + 1·1, 2·1 + 1·1] = [2, 1, 3]
- errors [0, 0, −3]; MSE = 9/3 = 3. Lesson: the third feature cannot be recovered from this particular W; training would change W to fix that.

B6.2 Shapes in a deep AE

Input batch 64 × 784. Layers 784 → 128 → 64 → 32 → 64 → 128 → 784. Output shapes: 64×128, 64×64, 64×32, 64×64, 64×128, 64×784. Weight matrices: 784×128, 128×64, 64×32, 32×64, 64×128, 128×784.

B6.3 Compression ratio

784 → 32 gives a compression ratio of 784/32 = 24.5 : 1. The 100 → 75 → 25 example is 4 : 1.

B6.4 Identity copy under denoising

x = [0.8, 0.3, 0.9, 0.5, 0.2], masked $\tilde{x}$ = [0.8, 0, 0.9, 0, 0.2]. If the network just copied $\tilde{x}$: errors [0, 0.3, 0, 0.5, 0] → squares [0, 0.09, 0, 0.25, 0] → MSE = 0.34/5 = 0.068 > 0. So pure copying is penalized; the network must guess the masked values from their neighbors/context.

B6.5 Dense vs conv parameter counts for the same task

Dense 784 → 64: 784×64 + 64 = 50,240 parameters. A 3×3 conv with 16 filters on a 1-channel image: 3×3×1×16 + 16 = 160 parameters. Conv layers share weights across positions, hence far fewer parameters and built-in locality.

B7. Convolution and pooling arithmetic (for the CNN autoencoder)

B7.1 Output size of a convolution

output = floor((n + 2p − k) / s) + 1, where n = input size, k = kernel size, p = padding, s = stride.

n	k	p	s	Output	Comment
28	3	1	1	28	this is padding='same' with a 3×3 kernel
28	3	0	1	26	padding='valid'
28	5	0	1	24	
28	3	0	2	13	floor(25/2) + 1
7	3	1	1	7	

In Keras, padding='same' with stride 1 always keeps the spatial size; padding='valid' means no padding.

B7.2 Number of parameters in Conv2D

params = k × k × (input channels) × (number of filters) + (number of filters for biases). Example: Conv2D(64, (3,3)) on a 1-channel image: 3×3×1×64 + 64 = 640. On 64 input channels with 128 filters: 3×3×64×128 + 128 = 73,856. Pooling and upsampling layers have 0 parameters.

B7.3 Max pooling

MaxPooling2D((2,2)) takes the maximum of each 2×2 window (stride 2 by default), halving height and width. Example: window [[1,3],[2,4]] → 4. Feature map 4×4:

```
[1 3 2 1]
[2 4 0 1]   -> pooled 2x2: [[4, 2],
[5 1 3 2]                  [5, 6]]
[0 2 6 1]
```

Top-left window max = 4; top-right window [[2,1],[0,1]] max = 2; bottom-left [[5,1],[0,2]] max = 5; bottom-right [[3,2],[6,1]] max = 6. With padding='same', odd sizes round up: 7 → ceil(7/2) = 4.

B7.4 Shape trace of the lecture's CNN AE, and element counts

Layer	Shape	Number of values
Input	28×28×1	784
conv1	28×28×64	50,176
pool1	14×14×64	12,544
conv2	14×14×128	25,088
pool2	7×7×128	6,272
latent	7×7×256	12,544
up1	14×14×256	50,176
conv	14×14×128	25,088
up2	28×28×128	100,352
conv	28×28×64	50,176
output	28×28×1	784

This table shows why the slides' claim of compression is about spatial size: the latent (12,544 values) is 16 times larger than the 784-pixel input in raw count. In practice such a CNN AE learns good local features but you would add a bottleneck (fewer channels, or a dense layer) or a regularizer if you want a truly compact code.

B8. Denoising autoencoder in depth

B8.1 Training procedure

1. Take a clean mini-batch x.
2. Generate corrupted copy $\tilde{x}$ by sampling noise (a NEW random corruption each time the sample is seen).
3. Forward: $h = g(W \tilde{x} + b)$, $\hat{x} = f(W' h + c)$.
4. Loss compares $\hat{x}$ with the CLEAN x: $L(x, \hat{x})$.
5. Backpropagate, update weights.

Fresh noise each epoch matters: if the same corrupted version were reused, the network could memorize the noise pattern. With new noise every time, the network can only succeed by learning the clean-data structure.

B8.2 Choosing the noise

Noise	Typical setting	Best for
Gaussian	σ around 0.1–0.5 for [0,1] data; clip result to [0,1]	natural images, sensor noise
Masking	q around 0.1–0.5	binary data, tabular data, robustness to missing values
Salt-and-pepper	fraction of pixels corrupted (e.g. 5–20%)	images with dead pixels / transmission errors

Too little noise: the DAE stays close to identity. Too much noise: little information survives, learning is hard and reconstruction is blurry. The noise level is a hyper-parameter, much like λ in CAE or β in SAE.

B8.3 Gaussian noise worked example

x = [0.50, 0.80, 0.20, 0.90], noise ε = [0.01, −0.05, 0.03, −0.08] → $\tilde{x}$ = [0.51, 0.75, 0.23, 0.82]. Sum check element by element: 0.50 + 0.01 = 0.51; 0.80 − 0.05 = 0.75; 0.20 + 0.03 = 0.23; 0.90 − 0.08 = 0.82. If the network just returned $\tilde{x}$: squared errors 0.0001, 0.0025, 0.0009, 0.0064 → sum 0.0099 → MSE 0.0025. Small but not zero; a DAE aims to push this below the noise floor by learning structure.

B8.4 Probability of a pixel being masked, in more detail

With q = 0.4 on d = 5 pixels, the number masked M follows a Binomial(5, 0.4) distribution: E[M] = 2, $P(M = 0) = 0.6^5 = 0.0778$, $P(M = 5) = 0.4^5 = 0.0102$. This is why "2 values masked" is an average, not a guarantee.

B8.5 Why neighbors help (the conditional-probability view)

The goal is to model $p(x_{ij} \mid N_{ij})$. A convolutional DAE does not count neighbors explicitly; the learned filters implement a learned function $F(N_{ij})$. Examples of what filters learn: stroke continuity (if both neighbors on a line are dark, the middle is probably dark), smoothness (in flat regions, a pixel equals its neighborhood average), edge continuation.

Quick neighborhood estimate summary:

- Binary: majority vote / fraction of 1s among 8 neighbors.
- Grayscale: mean (or median) of neighbors. Mean is sensitive to outliers; median is robust against salt-and-pepper noise. Example with salt-and-pepper outlier: neighbors {120,125,255,123,127,122,124,126}. Mean = 1122/8 = 140.25 (pulled up by the 255); median = sorted {120,122,123,124,125,126,127,255}, middle values 124 and 125 → 124.5. The median is much closer to the true local level.

B8.6 Keras code for a convolutional denoising AE

```
import numpy as np
from tensorflow.keras import layers, models
from tensorflow.keras.datasets import mnist

(x_train, _), (x_test, _) = mnist.load_data()
x_train = x_train.astype('float32') / 255.0
x_test  = x_test.astype('float32') / 255.0
x_train = x_train.reshape(-1, 28, 28, 1)
x_test  = x_test.reshape(-1, 28, 28, 1)

# corrupt (Gaussian noise), keep values in [0, 1]
noise = 0.3
x_train_noisy = np.clip(x_train + noise *
np.random.normal(size=x_train.shape), 0., 1.)
x_test_noisy  = np.clip(x_test  + noise *
np.random.normal(size=x_test.shape),  0., 1.)

inputs = layers.Input(shape=(28, 28, 1))
x = layers.Conv2D(32, (3, 3), activation='relu', padding='same')(inputs)
x = layers.MaxPooling2D((2, 2), padding='same')(x)          # 14x14x32
x = layers.Conv2D(64, (3, 3), activation='relu', padding='same')(x)
latent = layers.MaxPooling2D((2, 2), padding='same')(x)     # 7x7x64
x = layers.Conv2D(64, (3, 3), activation='relu', padding='same')(latent)
x = layers.UpSampling2D((2, 2))(x)                           # 14x14x64
x = layers.Conv2D(32, (3, 3), activation='relu', padding='same')(x)
x = layers.UpSampling2D((2, 2))(x)                           # 28x28x32
outputs = layers.Conv2D(1, (3, 3), activation='sigmoid', padding='same')(x)

dae = models.Model(inputs, outputs)
```

```
dae.compile(optimizer='adam', loss='binary_crossentropy')
# input = NOISY, target = CLEAN  (this line is the whole idea of a DAE)
dae.fit(x_train_noisy, x_train, epochs=10, batch_size=128,
        validation_data=(x_test_noisy, x_test))
```

The key line is `fit(x_train_noisy, x_train)`: noisy images in, clean images as the target. Fixed noise as written here is the simple version; to resample noise every epoch you can write a data generator or add a Keras GaussianNoise layer right after the input (it is only active during training).

B9. Sparse autoencoder in depth

B9.1 Why sparsity helps

Each hidden neuron becomes a specialist: it fires only for particular patterns (a stroke orientation, a corner, a texture). Because only a few neurons fire per input, the code is a sparse combination of specialists, which is easy to interpret and often useful for classification. Biological analogy used in the literature: only a small fraction of neurons in visual cortex respond to any given stimulus.

B9.2 Two ways to impose sparsity

	KL-divergence penalty (lecture)	L1 activity penalty
Penalty	$\Sigma_j \mathrm{KL}(\rho \parallel \hat{\rho}_j)$	$\lambda \Sigma_j \lvert h_j \rvert$
What it looks at	average activation of each neuron over the dataset/batch	activations for each sample
Needs sigmoid (activation in (0,1))?	Yes	No (works with ReLU)
Target	an explicit desired mean activation ρ	pushes activations toward 0

L1 example: h = [0, 0, 1.0, 0] has L1 = 1.0; h = [0.5, 0.5, 0.5, 0.5] has L1 = 2.0. The dense code pays double.

B9.3 Effect of the sparsity target ρ (guidance)

Smaller ρ (e.g. 0.005) → very strict sparsity; neurons are almost always off. Larger ρ (e.g. 0.1) → milder. β controls how heavily the whole penalty weighs against reconstruction. Too large β: reconstructions suffer (all neurons suppressed). Too small β: sparsity has no effect (back to the overcomplete identity problem).

B9.4 More KL values to build intuition (natural log)

Fix ρ = 0.05.

$\hat{\rho}$	$KL(0.05 \| \hat{\rho})$
0.05	0
0.10	$0.05 \cdot \ln(0.5) + 0.95 \cdot \ln(0.95/0.90) = -0.03466 + 0.95(0.05407) = -0.03466 + 0.05137 = 0.0167$
0.20	0.0940 (computed in Part A)
0.50	$0.05 \cdot \ln(0.1) + 0.95 \cdot \ln(1.9) = -0.11513 + 0.95(0.64185) = -0.11513 + 0.60976 = 0.4946$
0.01	$0.05 \cdot \ln(5) + 0.95 \cdot \ln(0.95/0.99) = 0.08047 + 0.95(-0.04124) = 0.08047 - 0.03918 = 0.0413$

Two observations: the penalty rises steeply as $\hat{\rho}$ grows well above ρ, and it is also positive when $\hat{\rho}$ is below ρ (0.0413 at $\hat{\rho} = 0.01$), so KL pulls the average toward ρ from both sides.

B9.5 Keras code: sparse AE with L1 activity regularizer, and a KL regularizer

```
from tensorflow.keras import layers, models, regularizers
import tensorflow as tf

# (a) L1 sparsity on the hidden activations
inputs = layers.Input(shape=(784,))
h = layers.Dense(128, activation='relu',
                 activity_regularizer=regularizers.l1(1e-5))(inputs)
outputs = layers.Dense(784, activation='sigmoid')(h)
sae_l1 = models.Model(inputs, outputs)
sae_l1.compile(optimizer='adam', loss='binary_crossentropy')

# (b) KL-divergence sparsity (sigmoid hidden units, rho = target mean
```

```
activation)
class KLSparsity(regularizers.Regularizer):
    def __init__(self, rho=0.05, beta=3.0):
        self.rho, self.beta = rho, beta
    def __call__(self, activations):
        rho_hat = tf.reduce_mean(activations, axis=0)      # mean over the
batch, per neuron
        rho_hat = tf.clip_by_value(rho_hat, 1e-7, 1 - 1e-7)
        kl = self.rho * tf.math.log(self.rho / rho_hat) + \
             (1 - self.rho) * tf.math.log((1 - self.rho) / (1 - rho_hat))
        return self.beta * tf.reduce_sum(kl)

h2 = layers.Dense(128, activation='sigmoid',
                  activity_regularizer=KLSparsity(0.05, 3.0))(inputs)
```

In (b), rho_hat is the batch estimate of the average activation, which is the practical version of $\hat{\rho}_j = (1/m)\Sigma\, a_j(x^{(i)})$.

B10. Contractive autoencoder in depth

B10.1 Geometric picture

The encoder maps the input space into hidden space. A contractive penalty makes this map "flat" (small slope) in directions that don't matter, so nearby inputs land on nearby codes. It does not shrink everything: directions along which the data genuinely varies (the data manifold) must still be represented well, otherwise reconstruction suffers. Net effect: sensitivity is kept along the data manifold and suppressed away from it. That is the "preserve important variations, suppress unimportant ones" trade-off on the slide.

B10.2 The Jacobian for a sigmoid layer

For $h = \sigma(Wx + b)$: $\partial h_l/\partial x_j = h_l\,(1 - h_l)\,W_{lj}$. Therefore

$\|J\|^2_F = \Sigma_l\,[h_l\,(1 - h_l)]^2 \cdot \Sigma_j\, W_{lj}^2$

This lets you compute the penalty without building the whole matrix: for each hidden unit take (its slope factor)² times the sum of squares of its incoming weights. Notice that both small weights and saturated units (h near 0 or 1, so h(1−h) near 0) lower the penalty.

B10.3 Worked example 2 (3 inputs, 2 hidden units)

W = [[1, 0, 2],[−1, 1, 0]] (row l = weights into hidden unit l). Suppose the hidden activations at the data point are h = [0.5, 0.8].

- Slope factors: h1(1−h1) = 0.25; h2(1−h2) = 0.8 × 0.2 = 0.16.
- Row sums of squares: row 1 = 1 + 0 + 4 = 5; row 2 = 1 + 1 + 0 = 2.
- $\|J\|^2_F = 0.25^2 \times 5 + 0.16^2 \times 2 = 0.0625 \times 5 + 0.0256 \times 2 = 0.3125 + 0.0512 = 0.3637$.
- The Jacobian itself: row 1 = 0.25·[1,0,2] = [0.25, 0, 0.5]; row 2 = 0.16·[−1,1,0] = [−0.16, 0.16, 0]. Sum of squares: 0.0625 + 0 + 0.25 + 0.0256 + 0.0256 + 0 = 0.3637. ✓
- With λ = 0.5 and L_rec = 0.08: L_CAE = 0.08 + 0.5×0.3637 = 0.2619.

B10.4 Interpreting entries

J[1,3] = 0.5 means: increasing x_3 by 0.1 changes h_1 by about 0.05 (first-order). J[2,2] = 0.16: increasing x_2 by 0.1 changes h_2 by about 0.016. h_1 is much more sensitive to x_3 than h_2 is to x_2.

B10.4b CAE vs plain weight decay

Weight decay penalizes $\|W\|^2$ directly. The contractive penalty penalizes the derivative of the whole encoder with respect to the input. For a sigmoid layer they are related (the formula contains ΣW^2) but the CAE penalty also includes the activation slope, so it behaves differently and acts on the function rather than only the parameters.

B10.5 CAE vs DAE

Both make the representation robust to small input changes. DAE does it empirically by training with corrupted inputs; CAE does it analytically by penalizing the derivative. For small noise, the two behave similarly: a DAE trained with small Gaussian noise effectively encourages a small Jacobian. CAE has an explicit knob (λ); DAE's knob is the noise level.

B10.6 Code sketch for a CAE (single sigmoid encoder layer)

```
import tensorflow as tf
from tensorflow.keras import layers, models

class CAE(models.Model):
    def __init__(self, n=784, k=64, lam=1e-4):
        super().__init__()
        self.enc = layers.Dense(k, activation='sigmoid')
```

```
        self.dec = layers.Dense(n, activation='sigmoid')
        self.lam = lam
    def call(self, x):
        h = self.enc(x)
        x_hat = self.dec(h)
        W = self.enc.kernel                                    # shape (n, k)
        dh = h * (1 - h)                                       # (batch, k)
        w_sq = tf.reduce_sum(tf.square(W), axis=0)             # (k,)  sum_j
W_jl^2
        penalty = tf.reduce_mean(tf.reduce_sum(tf.square(dh) * w_sq, axis=1))
        self.add_loss(self.lam * penalty)                      # lambda *
||J||_F^2
        return x_hat

cae = CAE()
cae.compile(optimizer='adam', loss='binary_crossentropy')
# cae.fit(x_train_flat, x_train_flat, epochs=20, batch_size=256)
```

The add_loss line is the term $\lambda\|J_x(h)\|^2_F$ added to the reconstruction loss, using the closed form from B10.2.

B11. Extra solved numericals (set 2)

N1. Parameter counts, dense AE $10 \to 4 \to 10$ (with biases)

- Encoder: 10×4 + 4 = 44
- Decoder: 4×10 + 10 = 50
- Total = 94 parameters.
- With tied weights (decoder weight = encoder weight transposed): weights shared, so 40 + 4 (encoder bias) + 10 (decoder bias) = 54.

N2. Tied-weight linear AE, $2 \to 1 \to 2$

x = [1, 1], W_e = [[1],[2]] (2×1). h = 1·1 + 1·2 = 3. $W_d = W_e^T$ = [1, 2]. $\hat{x}$ = 3·[1, 2] = [3, 6]. Errors [2, 5]; MSE = (4 + 25)/2 = 14.5.

N3. BCE, 3 features (natural log)

x = [1, 1, 0], $\hat{x}$ = [0.9, 0.6, 0.2].

- $-\ln(0.9) = 0.1054$

- −ln(0.6) = 0.5108
- −ln(1 − 0.2) = −ln(0.8) = 0.2231 Sum = 0.8393; mean over 3 features = 0.2798.

N4. Forward pass with biases and negative input

x = [1, −1]. Encoder W1 = [[1, 2],[−1, 0]], b1 = [0, −1], ReLU. Decoder W2 = [[0.5, −1],[1, 0.5]], linear, no bias.

- h1 = 1(1) + (−1)(−1) + 0 = 2 → 2
- h2 = 1(2) + (−1)(0) − 1 = 1 → 1
- x̂1 = 2(0.5) + 1(1) = 2
- x̂2 = 2(−1) + 1(0.5) = −1.5 Errors against x = [1, −1]: [1, −0.5]. MSE = (1 + 0.25)/2 = 0.625. (Note that a negative output is only possible because the output is linear; a sigmoid could never return −1.5.)

N5. Sparse AE total loss

L_rec = 0.2, β = 3, KL penalties of the neurons: 0.05 and 0.02. Total = 0.2 + 3 × (0.05 + 0.02) = 0.2 + 0.21 = 0.41.

N6. Transposed convolution, all-ones input

Input [[1,1],[1,1]], kernel [[1,2],[3,4]], stride 1 → 3×3. Place the kernel (times 1) at each of the four input positions and add:

- (0,0): rows 0–1, cols 0–1
- (0,1): rows 0–1, cols 1–2
- (1,0): rows 1–2, cols 0–1
- (1,1): rows 1–2, cols 1–2

Result:

```
[1   3   2]
[4  10   6]
[3   7   4]
```

Check: centre = 4 + 3 + 2 + 1 = 10 (all four placements overlap there). Sum of all entries = 40 = (sum of input 4) × (sum of kernel 10). ✓ Notice the uneven pattern (1, 3, 2 / 4, 10, 6 / 3, 7, 4) even though the input is perfectly uniform: this is the root of checkerboard artifacts.

N7. Average activations from a small table

Activations of two neurons over 3 samples: [[0.1, 0.5],[0.0, 0.7],[0.2, 0.6]]. $\hat{\rho}_1 = (0.1 + 0.0 + 0.2)/3 = 0.1$; $\hat{\rho}_2 = (0.5 + 0.7 + 0.6)/3 = 0.6$. With $\rho = 0.1$, neuron 1 has zero penalty and neuron 2 has a large one.

N8. Mean vs median for a salt outlier

Neighbors {10, 12, 11, 200, 13, 12, 11, 10}. Sum = 279, mean = 34.875. Sorted: 10, 10, 11, 11, 12, 12, 13, 200 → median = (11 + 12)/2 = 11.5. Median is the better estimate under salt-and-pepper noise.

N9. A convolution with a vertical-edge kernel

Patch [[1,2,3],[4,5,6],[7,8,9]], kernel [[1,0,−1],[1,0,−1],[1,0,−1]]. Result = (1 + 4 + 7) − (3 + 6 + 9) = 12 − 18 = −6 (the middle column is multiplied by 0; this kernel responds to horizontal intensity change).

N10. KL in base 10 versus natural log

$\rho = 0.05$, $\hat{\rho} = 0.2$: natural log gives 0.0940; base 10 gives $0.0940 \times 0.4343 = 0.0408$. Always check which base the question expects.

N11. Reconstruction quality comparison

Two AEs reconstruct x = [0.5, 0.5, 0.5, 0.5]. AE-A gives [0.5, 0.5, 0.5, 0.5], AE-B gives [0.6, 0.4, 0.6, 0.4]. MSE_A = 0; MSE_B = $4 \times 0.01 / 4 = 0.01$. Lower MSE = better reconstruction, but remember the limitation: it does not tell you which latent code is more useful for downstream tasks.

N12. How many values are masked? (Binomial)

6 features, q = 0.5. Expected masked = 3. P(exactly 0 masked) = $0.5^6 = 0.0156$ (1/64). P(all 6 masked) = 1/64.

B12. Question bank 2 – MCQs (40 more)

B1. A linear autoencoder (no nonlinear activations) trained with MSE and k < n hidden units learns the same subspace as: A) ICA B) PCA C) K-means D) t-SNE **Answer: B.**

B2. For a sigmoid output with BCE loss, ∂L/∂(pre-activation) simplifies to: A) $x - \hat{x}$ B) $\hat{x} - x$ C) $\hat{x} \cdot x$ D) $1 - \hat{x}$ **Answer: B.**

B3. The derivative of ReLU at an input of −1.5 is: A) 1 B) 0 C) −1 D) 0.5 **Answer: B.**

B4. The maximum value of the sigmoid derivative $\sigma'(x)$ is: A) 1 B) 0.5 C) 0.25 D) 0.1 **Answer: C.** At x = 0: 0.5 × 0.5 = 0.25.

B5. A convolution on a 28×28 image with a 5×5 kernel, stride 1, no padding gives: A) 28×28 B) 26×26 C) 24×24 D) 23×23 **Answer: C.** 28 − 5 + 1 = 24.

B6. Number of parameters in Conv2D(16, (3,3)) applied on a 3-channel input: A) 144 B) 432 C) 448 D) 160 **Answer: C.** 3×3×3×16 + 16 = 448.

B7. Parameters in Dense(64) applied to a 784-dimensional input: A) 50,176 B) 50,240 C) 784 D) 848 **Answer: B.** 784×64 + 64.

B8. MaxPooling2D((2,2)) on the window [[7,2],[3,9]] returns: A) 5.25 B) 7 C) 9 D) 3 **Answer: C.**

B9. Transposed convolution of a 4×4 input with a 3×3 kernel, stride 1, no padding, gives: A) 4×4 B) 5×5 C) 6×6 D) 7×7 **Answer: C.** 4 + 3 − 1 = 6.

B10. Transposed convolution of a 4×4 input, 3×3 kernel, stride 2, no padding gives: A) 8×8 B) 9×9 C) 10×10 D) 11×11 **Answer: B.** (4 − 1)×2 + 3 = 9.

B11. UpSampling2D((3,3)) applied to a 5×5 feature map gives: A) 8×8 B) 10×10 C) 15×15 D) 125×125 **Answer: C.**

B12. The number of values in a 7×7×256 latent tensor is: A) 1,792 B) 12,544 C) 784 D) 3,136 **Answer: B.** 49 × 256.

B13. A Jacobian J = [[1, 2],[0, −2]]. $\|J\|^2_F$ is: A) 5 B) 7 C) 9 D) 3 **Answer: C.** 1 + 4 + 0 + 4.

B14. L1 activity penalty with λ = 0.1 and h = [0.5, 0.5, 0.5, 0.5]: A) 0.05 B) 0.1 C) 0.2 D) 2.0 **Answer: C.** 0.1 × 2.0.

B15. Why should noise be re-sampled each epoch in a DAE? A) To speed up training B) So the network cannot memorize one fixed corruption C) To reduce the latent size D) To avoid using a loss **Answer: B.**

B16. Which estimator is more robust to a salt-and-pepper outlier among neighbors? A) Mean B) Median C) Sum D) Maximum **Answer: B.**

B17. In the Keras DAE code, why use np.clip after adding Gaussian noise? A) To make noise larger B) To keep pixel values in [0,1] C) To normalize the labels D) To reduce the number of epochs **Answer: B.**

B18. Five pixels, masking probability q = 0.3, independent. P(no pixel masked) = A) 0.3^5 B) $0.7^5 \approx 0.168$ C) 5 × 0.3 D) 0.5 **Answer: B.**

B19. With ρ = 0.05, which $\hat{\rho}$ yields the largest KL penalty? A) 0.05 B) 0.10 C) 0.20 D) 0.50 **Answer: D.** KL values: 0, 0.017, 0.094, 0.495.

B20. Sigmoid hidden units with activations near 0 or 1 (saturated) make the CAE Jacobian penalty: A) Larger B) Smaller C) Unchanged D) Undefined **Answer: B.** The slope factor h(1 − h) approaches 0.

B21. If all weights of a sigmoid encoder are doubled (and activations were hypothetically unchanged), the CAE penalty becomes about: A) 2× B) 4× C) 0.5× D) unchanged **Answer: B.** It depends on ΣW^2, which scales by 2^2.

B22. With tied weights, the gradient for W_e is: A) Only from the encoder path B) Only from the decoder path C) The sum of encoder-path and decoder-path contributions D) Zero **Answer: C.**

B23. Compression ratio of 784 → 32 is: A) 12.25 B) 24.5 C) 32 D) 49 **Answer: B.**

B24. In the deep AE 784-128-64-32-64-128-784, the number of weight matrices is: A) 5 B) 6 C) 7 D) 3 **Answer: B.** One between each pair of consecutive layers.

B25. If λ = 0 in a CAE, the model reduces to: A) A denoising AE B) A standard AE C) A VAE D) A sparse AE **Answer: B.**

B26. If β = 0 in a sparse AE, the model reduces to: A) A standard AE (with whatever bottleneck it has) B) A contractive AE C) A VAE D) A CNN **Answer: A.**

B27. Which layer has learnable parameters? A) UpSampling2D B) MaxPooling2D C) Conv2DTranspose D) Flatten **Answer: C.**

B28. A 28×28 image flattened for a dense AE has how many inputs? A) 28 B) 56 C) 784 D) 1,568 **Answer: C.**

B29. An AE trained only on normal data is used for anomaly detection. An anomaly is expected to have: A) Very low reconstruction error B) High reconstruction error C) Zero latent code D) Negative loss **Answer: B.** The model reconstructs unseen kinds of data poorly.

B30. For a sparse AE using KL divergence, $\hat{\rho}_j$ is computed as: A) Average over the neurons for one sample B) Average of neuron j over training samples C) The maximum activation D) The weight norm **Answer: B.**

B31. With a ReLU hidden layer, why is the KL-sparsity formula awkward? A) ReLU outputs can exceed 1 or equal 0 so $\hat{\rho}$ may fall outside (0,1) and logs fail B) ReLU is slow C) ReLU is linear D) ReLU has no derivative **Answer: A.**

B32. Compared with an undercomplete AE, which statement about an overcomplete AE is correct? A) It cannot be trained B) Without regularization it may learn identity and fail to extract features C) It always generalizes better D) It has fewer parameters **Answer: B.**

B33. Which noise preserves the possibility that a corrupted pixel looks perfectly valid (a plausible value)? A) Salt-and-pepper (0 or max) B) Gaussian noise of small σ C) Both are always obviously invalid D) None **Answer: B.** Small Gaussian noise gives plausible but wrong values; salt/pepper values are extreme.

B34. The sum of all entries of a transposed convolution output (stride 1, no padding) equals: A) Sum of input × sum of kernel B) Sum of input + sum of kernel C) Max of input × max of kernel D) Zero **Answer: A.**

B35. Why might a CNN autoencoder reconstruct images better than a dense AE of similar size? A) It uses more labels B) Convolutions exploit local spatial structure and share weights C) It has no decoder D) It uses no activation **Answer: B.**

B36. Which statement describes the CAE trade-off? A) Preserve all variation B) Preserve important variations and suppress unimportant ones C) Remove all variation D) Make hidden codes random **Answer: B.**

B37. What does a very large β do in a sparse AE? A) Improves reconstruction B) Over-suppresses activations, hurting reconstruction C) Makes network deeper D) Nothing **Answer: B.**

B38. What does a very large λ do in a CAE? A) Makes the hidden code nearly constant, losing detail B) Makes it more sensitive C) Increases latent size D) Nothing **Answer: A.**

B39. The VAE fixes the plain AE's inability to generate by: A) Removing the decoder B) Regularizing the latent space with a KL term so it can be sampled C) Using only linear layers D) Training on labels **Answer: B.**

B40. In the Lecture 16 numerical example, the number of latent values is: A) 5 B) 4 C) 3 D) 2 **Answer: C.** 5 → 4 → 3 → 4 → 5.

B13. Multiple-select questions (more than one answer can be correct)

M1. Which of these are regularized autoencoder variants? A) Denoising B) Sparse C) Contractive D) Deep **Answer: A, B, C.** Deep AE is an architecture (more layers), not a regularizer.

M2. Which statements about upsampling and transposed convolution are true? A) Nearest-neighbor upsampling has no parameters B) Transposed convolution is learnable C) Transposed convolution can produce checkerboard artifacts D) Upsampling + convolution is a common fix for the artifacts **Answer: A, B, C, D.**

M3. Reasons the input is corrupted in a DAE: A) Prevent trivial identity copying B) Force use of context C) Increase the latent size D) Learn robust features **Answer: A, B, D.**

M4. Limitations of a standard AE: A) Low reconstruction error does not guarantee useful features B) Cannot generate new samples C) Latent size choice is critical D) Cannot be trained by gradient descent **Answer: A, B, C.**

M5. Which are valid ways to impose sparsity? A) KL divergence on mean activations B) L1 penalty on activations C) Adding noise to the input D) Jacobian penalty **Answer: A, B.** C is denoising; D is contractive.

B14. True / False

1. In an AE the number of output neurons equals the number of input features. **True.**
2. An overcomplete AE always learns better features than an undercomplete one. **False.**
3. A DAE reconstructs the noisy input. **False** (reconstructs the clean input).
4. In a sparse AE the penalty is computed from a single sample's activations. **False** (average over samples).
5. $KL(\rho \| \hat{\rho})$ is zero when $\hat{\rho} = \rho$. **True.**
6. CAE penalizes the weights only. **False** (it penalizes derivatives of h with respect to x).
7. Higher λ in CAE gives more stable features. **True.**
8. Nearest-neighbor upsampling has learnable parameters. **False.**
9. Padding='same' with stride 1 keeps the spatial size unchanged. **True.**
10. Tied weights double the number of parameters. **False** (they halve the weight count).
11. Standard AEs can be used directly to generate new, diverse samples. **False.**
12. A linear AE with MSE learns PCA's subspace. **True.**

13. BCE is suited for real-valued unbounded data. **False** (use MSE/linear).
14. Sigmoid at the output keeps reconstructed pixels within [0,1]. **True.**
15. The Jacobian of the hidden layer with respect to the input has size k × n (k hidden, n input). **True.**

B15. Fill in the blanks

1. The smaller internal representation produced by the encoder is called the _____. (latent code)
2. If dim(h) < dim(x) the AE is _____. (undercomplete)
3. For binary inputs the decoder uses _____ activation and _____ loss. (sigmoid; binary cross-entropy)
4. Tied weights mean W_d = _____. (W_e^T)
5. In a DAE, the noise type that sets pixels to 0 with probability q is _____. (masking noise)
6. _____ and _____ are the two extreme values used in salt-and-pepper noise. (0/black and 255/white)
7. The sparsity penalty uses the _____ divergence between ρ and $\hat{\rho}_j$. (KL / Kullback–Leibler)
8. The CAE penalty is the squared _____ norm of the _____. (Frobenius; Jacobian)
9. The artifact caused by uneven kernel overlap in transposed convolution is the _____ problem. (checkerboard)
10. _____ autoencoders address the limitations of standard AEs by regularizing the latent space. (Variational)

B16. Assertion–Reason questions

For each pair choose: (a) both true and R explains A; (b) both true but R does not explain A; (c) A true, R false; (d) A false, R true.

AR1. A: A DAE learns more robust features than a plain overcomplete AE. R: Corrupting the input removes the shortcut of direct copying. **(a).**

AR2. A: Low reconstruction error guarantees that the latent code is useful for classification. R: A plain AE can reconstruct well without its latent code clearly encoding semantic factors such as digit identity. **(d):** A is false, R is true.

AR3. A: A sparse AE with a large hidden layer can still learn useful features. R: The KL penalty keeps most neurons inactive for most inputs. **(a).**

AR4. A: Doubling the number of filters in the encoder's last conv layer always compresses the data more. R: More filters give a larger latent tensor. **(d):** A is false (it increases the latent size), R is true.

AR5. A: Upsampling followed by convolution can avoid checkerboard artifacts. R: Upsampling spreads values uniformly so the convolution kernel overlaps evenly. **(a).**

B17. Match the following

Technique	Key idea
1. Denoising AE	(c) Corrupt input, reconstruct clean
2. Sparse AE	(a) Few neurons active, KL penalty
3. Contractive AE	(d) Small Jacobian, stable representation
4. Undercomplete AE	(b) Bottleneck narrower than input
5. VAE	(e) Regularize latent space with KL for generation

Loss / activation pair	Data
MSE + linear	Real-valued, standardized
BCE + sigmoid	Binary or [0,1]
KL on mean activations	Sparsity
Frobenius norm of Jacobian	Contractivity

B18. How to write exam answers (scoring tips)

- Always start numerical answers by stating the formula, then substitute, then compute, then state the result with units/shape.
- For forward passes, write each neuron's calculation on its own line (as the lecture does) and say which column of W you used.
- For loss questions state which loss and why (data type → output activation → loss).

- For "explain" questions, use a diagram: x → encoder → h → decoder → $\hat{x}$ with dimensions labelled.
- For regularization questions, structure the answer as: problem (identity mapping) → idea → formula → effect of the hyper-parameter → example.
- For comparison questions, a table (as in section 9.4) earns marks quickly.
- State the log base if you compute a KL value.

B19. One-page master summary

1. AE = encoder + latent code + decoder, trained to reproduce its input. Unsupervised; target is the input.
2. Undercomplete (dim h < dim x) forces compression; overcomplete risks identity.
3. Linear AE: $H = X W_e$, $\hat{X} = H W_d$, tied weights $W_d = W_e^T$; shapes 1000×6 → 1000×3 → 1000×6.
4. Binary data: sigmoid + BCE. Real data: linear + MSE.
5. Types: shallow (one hidden layer), deep (many layers), CNN (conv + pool; upsample or transposed conv).
6. Upsampling: no parameters. Transposed conv: learnable, checkerboard problem. Upsample + conv: fix.
7. Regularized: denoising (corrupt input), sparse (KL on mean activation), contractive (Jacobian norm).
8. Numerical example: 5→4→3→4→5 gives z = [0.760, 0.714, 0.944], $\hat{x}$ = [0.67838, 0.7052, 0.69266, 0.68194, 0.64202], MSE = 0.07379.
9. Limitations: low error ≠ good features; no guarantee for downstream tasks; cannot generate; latent size critical.
10. VAE fixes this by adding a KL regularizer to the latent space.

Good luck with the exam. If you tell me the exact question formats or marks your NPTEL exam uses (for example 1-mark vs 2-mark MCQs, number of numericals), I can tune the practice set to match.